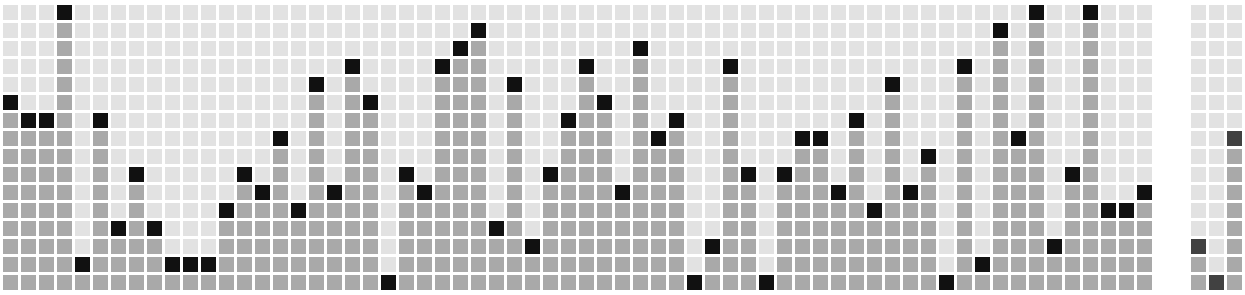
Noised: the quantum security layer for Solana

A security layer with four products: a launchpad where no key can rug a launch and the first trade is one fair price for everyone, a vault whose keys are hash-based, private balances, and a verifier for QSL-1, the token standard that names these guarantees. The creator's identity and long-term custody rest on hash-based signatures that a quantum computer cannot forge. With every derivation, every bound, and an honest list of what is still trusted.

SECTIONS	FIGURES, COMPUTED LIVE	TABLES	REFERENCES
15	20	13	27



Frontispiece. The 67 hash chains an LM-OTS signature over this paper's title would reveal: 64 digits of SHA-256(title), then the 3 checksum digits. Section 4 explains every pixel.

CONTENTS

01 Abstract and contributions
02 Threat model
03 System architecture
04 Hash-based signatures
05 Post-quantum creator identity, attestation and on-chain metadata
06 The fair-open batch
07 Contention and liveness
08 Launch Guard and the transfer hook
09 Liquidity and fees
10 The PQ vault (v4)
11 Private balances
12 Security evaluation
13 Performance
14 Limitations and future work
15 References

01 Abstract and contributions

Noised is a token launchpad on Solana built so that nobody can rug a launch, not even its creator, and so that the guarantees that matter still hold against an adversary with a cryptographically relevant quantum computer. Every privileged role of a launch is held by a program-derived address that has no private key. A pro-rata *fair-open batch* makes the first trade of every launch at one price for everyone who joined it. A transfer hook, Launch Guard, refuses every buy before that batch and enforces bounded anti-sniping rules for at most 72 hours, and graduated liquidity is locked forever. Post-quantum cryptography enters exactly where an elliptic-curve break would otherwise be fatal: a launch goes live only after its creator's hash-based identity (LMS, RFC 8554, height 10) signs the launch's exact parameters, its metadata and its opening window, verified on-chain; the token's metadata itself (name, symbol, description and icon) is a record stored on-chain at the address of its own hash, so that signature covers every byte a reader will display; and the PQ vault keeps tokens behind one-time LM-OTS keys that rotate in place and whose signatures, once revealed, always execute. This paper gives the threat model, the protocol, derivations of every bound the design relies on, the security argument for each component, measured compute and size costs, and an explicit list of what is still trusted. Nothing in Noised makes Solana itself post-quantum.

C1

Post-quantum origin, enforced by consensus

An LMS identity with an on-chain one-time-leaf bitmap signs a 189-byte message binding the launch, its parameters, the hash of its on-chain metadata record and its window; `launch` refuses to run without it, and every valid signature lands to burn its leaf (§5).

C2

PQ custody that never forces a second signature

Vault withdrawals read no account a third party controls, reserve instead of paying, and derive the LM-OTS randomizer deterministically, so a revealed signature always executes and re-signing is idempotent (§4, §10).

C3

A Sybil-neutral fair open

Pro-rata fills with proved payout bounds: splitting wallets, committing early or bundling with the launch gain nothing; oversubscription refunds proportionally (§6).

C4

A shield that admits exactly one buy

Until the batch, the hook passes only the buy made inside a top-level `execute_batch`, read from the instructions sysvar; the batch must also be the pool's first swap or it refunds (§8).

C5

Liveness quantified, safety unconditional

A write-lock contention cost model sets the commit and execution windows; a censored batch fails safe into a full refund (§7).

C6

Evidence

Eight internal reviews by AI agents, three of them multi-agent adversarial rounds, with regression tests against mainnet binaries, mutation testing of every program fix, and measured compute for every instruction (§12, §13).

STATUS

The programs have had internal adversarial reviews by AI agents (§12), and they stay upgradeable until they are finalized. Audit round 3's two program fixes (F1, F2) are merged, with their regression tests and mutants. Numbers tagged `FINAL BUILD` come from the final programs with on-chain metadata (2026-10-09); `V2 BUILD` from the protocol-v2 integration build of the same day (post-quantum attestation and the round-2 fixes, before the metadata change), for instructions the metadata change did not touch; `DOCS` are the published round-1b measurements of 2026-10-08. Values tagged `MODEL` are computed from stated formulas, not measured.

02 Threat model

We consider three adversaries of increasing strength and design for the strongest one wherever the protocol can. They are summarized in Table 1; the rest of the paper refers to them as $\mathcal{A}_{\mathrm{cl}}$, $\mathcal{A}_{\mathrm{q}}$ and $\mathcal{A}_{\mathrm{si}}$.

TABLE 1 Adversary classes.

ADVERSARY	CAPABILITIES	CANNOT
$\mathcal{A}_{\mathrm{cl}}$ classical	Any number of wallets; submits any transaction; reads every account; pays priority fees; runs bots and bundles.	Break Ed25519 or SHA-256; control which leader produces a block.
$\mathcal{A}_{\mathrm{q}}$ quantum	Everything above, plus a cryptographically relevant quantum computer: discrete logarithms in polynomial time (Shor), square-root speed-ups on search (Grover).	Invert SHA-256 faster than about 2^128 serial iterations; forge a hash-based signature it has not seen.
$\mathcal{A}_{\mathrm{si}}$ "superintelligence"	Everything above, plus perfect bug finding (it knows every reachable state of every program), optimal ordering and MEV strategy, unbounded coordination, and social engineering of any human who holds a key.	Violate an invariant that holds in every reachable state; break the hash function beyond generic bounds.

2.1 What a quantum computer breaks, and what it does not

Ed25519 [6] works in a group of prime order $\ell = 2^{252} + 27742317777372353535851937790883648493$ generated by a base point B. A public key is a point $A = [a]B$, and on Solana an ordinary account's address *is* that public key. Shor's algorithm [8] computes the discrete logarithm

$$a \equiv \log_B A \pmod{\ell} \tag{2.1}$$

in time polynomial in $\log \ell$; published 2026 estimates put a 256-bit elliptic-curve break at about 1,200 logical qubits [15], and NIST plans to disallow quantum-vulnerable signatures such as EdDSA after 2035 [14]. Knowing a is enough to produce valid signatures on any message. Because addresses are public keys, every funded wallet is exposed *at rest*, including receive-only addresses, and Solana has no protocol-level key rotation. We call this **harvest now, forge later**: unlike encrypted traffic, nothing has to be intercepted today. The public keys are already on chain, and the day a quantum computer exists every key behind them can sign. The same holds for every upgrade authority and multisig member key that can change a program, which is why §12 lists them as trusted parties.

Hash functions lose only to generic search. Grover's algorithm [9] finds a preimage of an n-bit hash in about

$$\frac{\pi}{4} 2^{n/2} \approx 2^{127.65} \quad \text{iterations for } n = 256, \tag{2.2}$$

and the iterations are long and serial[7]. For collisions, the best quantum algorithms do not beat classical parallel collision search once their hardware cost is counted [11], so SHA-256 keeps about 128 bits there too. Program-derived addresses are SHA-256 outputs that are not curve points: no private key exists, so there is nothing to recover, provided the program that owns them is immutable and trusts no Ed25519 authority. Fig. 1 puts these numbers side by side; LM-OTS and LMS inherit SHA-256's bound because, as §4 shows, forging them reduces to a 256-bit search[2].

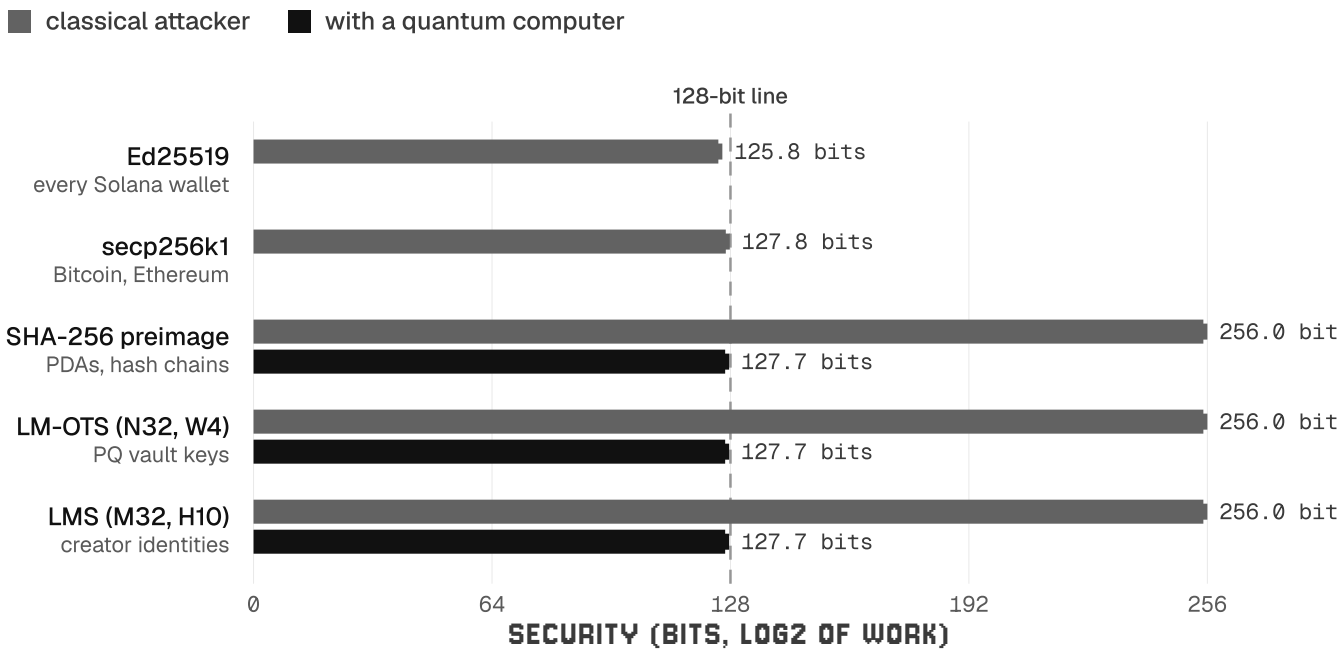


FIG. 1 **Generic attack cost against each primitive, classical and quantum.** log2 of the expected work of the best generic attack[1]. Elliptic-curve keys fall from about 126-128 bits (Pollard rho [10], $\sqrt{\pi \ell / 4}$) to polynomial time under Shor's algorithm; SHA-256 and the hash-based schemes built on it keep about 127.7 bits under Grover's search, $(\pi / 4)\, 2^{n/2}$ iterations for $n = 256$. Computed from the group orders and the formulas, not typed in.

2.2 The superintelligence adversary

No system can be proven safe against an arbitrarily capable adversary, and Noised makes no such claim. We take the goal to mean: leave $\mathcal{A}_{\text{si}}$ nothing to work with. Let the *trust surface* $\mathcal{T}(L)$ of a launch L be the set of keys, people and upgradeable programs whose action can change L's outcome after it starts. Perfect bug finding means any property that fails in *some* reachable state will be made to fail; social engineering means every element of $\mathcal{T}(L)$ that a human controls will eventually act against the user. The design rules follow:

No admin keys. The `Protocol` account is written once; no instruction pauses, updates or redirects anything. After `launch`, every DBC role (pool creator, partner, fee claimer, leftover receiver, LP owner) is a PDA.

Immutability. All four Noised programs are to be finalized (upgrade authority set to none) after independent audit; until then the upgrade key is the largest element of $\mathcal{T}$, and we say so.

Invariants, not procedures. Every guarantee is a property checked by the program in the instruction that could break it (post-conditions on the fresh mint, payout bounds, the first-swap check), so it holds in every reachable state, not only along the intended path.

Sybil neutrality. Mechanisms must not reward splitting capital across wallets or arriving first; the batch is additive in commits (§6). Per-wallet caps exist but are documented as limiting only naive concentration.

Safe failure under censorship. Liveness cannot be guaranteed against leaders and ordering gateways; every time-bounded step therefore fails into a full refund, never into a loss (§7).

Nothing off-chain is needed for safety. Origin, parameters, attestation and the token's metadata are stored and checked on-chain; verifiers re-read live state rather than trusting a certificate.

> **REMARK 2.1** *(what Noised does not claim).*
>
> Solana consensus (Ed25519 validator and vote keys), Token-2022, the associated-token-account program, and Meteora's DBC and DAMM v2 are upgradeable or classical, and remain in every launch's trust surface. Tokens held in ordinary wallets are quantum-vulnerable. Noised narrows the trust surface to these parties, discloses them in the app and in the verifier, and offers post-quantum custody (the vault) for value that must survive them. It does not make any token "quantum-proof".

03 System architecture

Noised is four Anchor programs and a TypeScript SDK (Fig. 2). The launchpad creates a Meteora Dynamic Bonding Curve (DBC) [18] configuration with fixed safe values, launches the pool with a Token-2022 [20] mint whose transfer hook is Launch Guard, runs the fair-open batch, routes fees by immutable rules and writes an on-chain certificate. At graduation DBC migrates the pool to DAMM v2 [19], where 100% of the liquidity is permanently locked under a launchpad PDA. The vault and the shield are independent of launches: post-quantum custody for any eligible mint, and opt-in confidential balances [23].

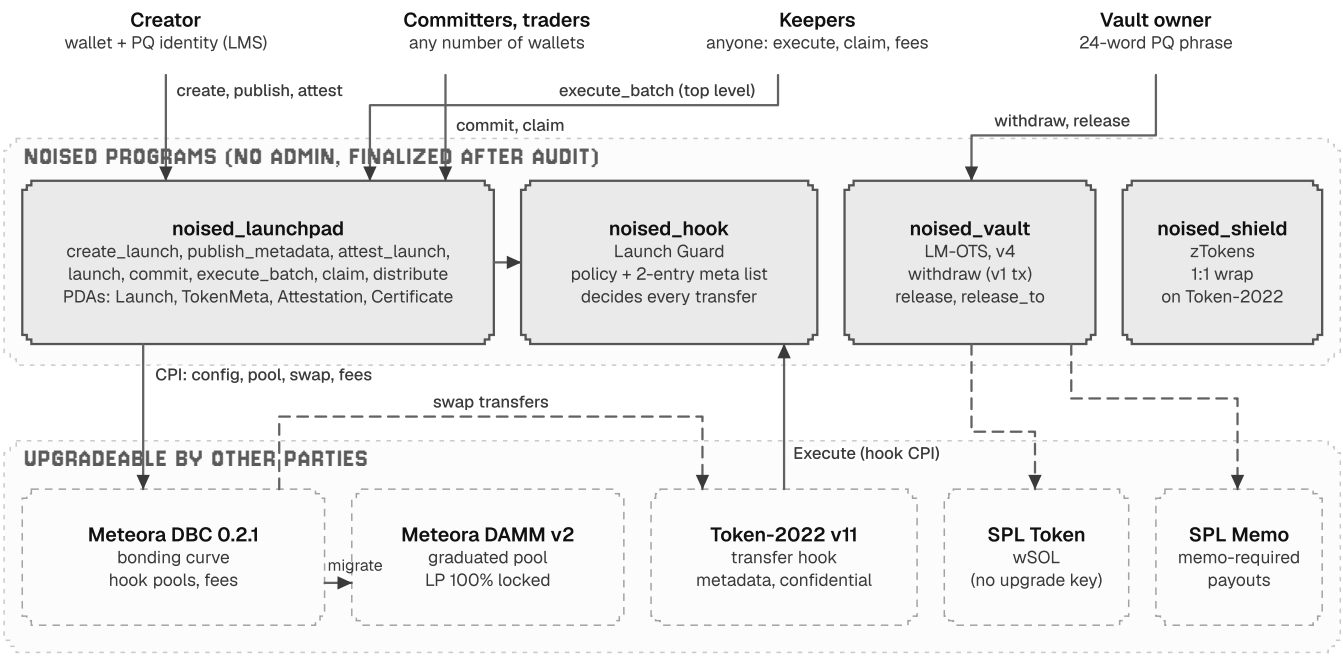


FIG. 2 **Architecture.** Solid boxes are Noised's four programs, which hold every launch role through PDAs and are to be finalized after audit. Dashed boxes are programs other parties can upgrade (§12). Arrows are instructions (from people) and cross-program invocations.

3.1 Roles held by addresses without keys

A program-derived address is $\mathrm{PDA}(s, \pi) = \mathrm{SHA256}(s \,\|\, \pi \,\|\, \texttt{"ProgramDerivedAddress"})$ for seeds s and program π, accepted only when the result is not an Ed25519 point; only π can sign for it. Table 2 lists every PDA a launch uses. None is ever closed, and no instruction changes a launch's fee recipients, hook policy or DBC configuration after creation.

TABLE 2 Launchpad PDAs and their roles.

ACCOUNT	SEEDS	ROLE
Protocol	`"protocol"`	Treasury and platform fee share, written once, never updatable.
Launch	`"launch", creator, seed[32]`	All creator choices (including `pq_identity`), timeline, batch caps and totals.
Launch authority	`"authority", launch`	DBC pool creator, partner, fee claimer and leftover receiver; owns the DAMM v2 LP position and the fee vaults.
Batch authority	`"batch", launch`	Owns the batch escrows and pays for the batch swap.
DBC config	`"dbc-config", launch`	The DBC transfer-hook configuration, created by CPI with this PDA as signer.
Hook authority	`"hook-authority"`	The only signer Launch Guard accepts for policies and for ending the shield.
Identity	`"identity", pq_identity`	A creator's LMS public key and its 1,024-bit used-leaf bitmap (§5).
Token metadata	`"metadata", launch, metadata_hash`	The token's name, symbol, uri mode and document, at the address of its own hash; never modified or closed (§5).
Attestation	`"attestation", launch`	The LMS signature that let the launch go live, kept in full, with the record's hash and the window.
Certificate	`"certificate", mint`	Immutable record of the launch checks, the policy and the post-quantum origin.
Commit	`"commit", launch, owner`	One participant's batch commitment and its fixed payout owners.
Payout vault	`"payout-vault", launch, vault`	What every commit naming one PQ vault committed, held to one wallet's cap together (§6).

3.2 Life of a launch

Fig. 3 follows one launch. `create_launch` validates the creator's parameters against the safety policy and creates the DBC configuration (status Configured). A throwaway key publishes the token's metadata record (`publish_metadata`); the creator's post-quantum identity then signs the launch's attestation message, which names that record by its hash, and the same key relays it in a version-1 transaction (`attest_launch`), which burns one LMS leaf. `launch`, permissionless inside the signed window and without arguments, creates the pool and the mint from the record, the hook's meta list and policy, verifies the mint's post-conditions and writes the certificate. With launch time T_0, commit window $\delta \in [60, 1800]$ s, execution window $E = 600$ s and guard $g \leq 72$ h, the timeline is

$$\underbrace{[T_0,\, T_0+\delta)}_{\text{commit}} \;\rightarrow\; \underbrace{[T_0+\delta,\, T_0+\delta+E)}_{\text{execute (ends early at the batch)}} \;\rightarrow\; \underbrace{[\,\cdot\,,\, T_0+g)}_{\text{guard}} \;\rightarrow\; \text{open} \;\rightarrow\; \text{graduation} \tag{3.1}$$

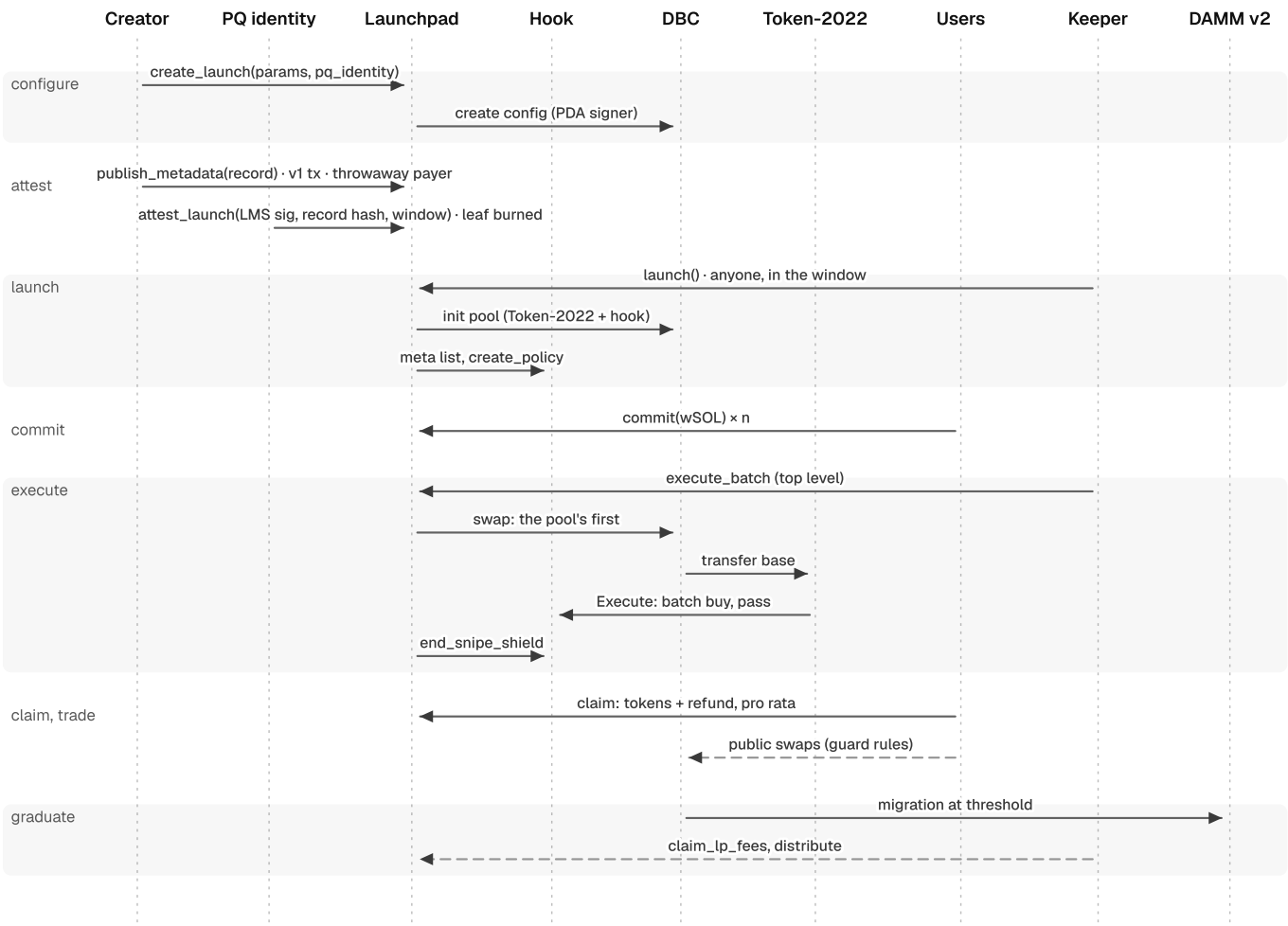


FIG. 3 **One launch, from configuration to graduation.** Time runs downward. Only the creator's hash-based identity can attest; anyone may publish the record, relay that signature, run `launch` inside the signed window, execute the batch, crank claims and route fees. Until the batch the hook refuses every buy but the batch's own (§8).

3.3 Call depth

Solana allows an instruction stack of depth 5. The deepest Noised path is the batch buy, `execute_batch` → DBC `swap` → Token-2022 → Launch Guard, of depth 4. The design never adds a layer in between; a `execute_batch` invoked by another program would reach depth 5 but is refused by the hook anyway (§8). The launchpad links neither the vault nor the shield: it recognizes a PQ vault (as treasury or payout owner) by the vault account's owner, discriminator, version and mint bytes alone.

04 Hash-based signatures

Both post-quantum components of Noised use the Leighton-Micali schemes of RFC 8554 [1]: the one-time signature LM-OTS with parameter set `LMOTS_SHA256_N32_W4` for vault keys, and the many-time tree LMS `LMS_SHA256_M32_H10` over it for creator identities. Their only assumption is SHA-256; their verification uses only the `sol_sha256` syscall. This section derives the parameters, proves the combinatorial core of their security, and quantifies why a one-time key must sign once.

4.1 Winternitz chains

Fix a key identifier I (16 bytes) and a leaf index q. Every hash is tweaked with its position, so work spent against one key, chain or step never helps against another:

$$H_{i,j}(x) = \mathrm{SHA256}\Big(I \,\|\, u32(q) \,\|\, u16(i) \,\|\, u8(j) \,\|\, x\Big), \qquad c_i^{a \to b}(x) = H_{i,b-1} \circ \cdots \circ H_{i,a}(x). \tag{4.1}$$

With hash length $n = 32$ bytes and Winternitz width $w = 4$ bits, RFC 8554 §4.1 gives the number of message digits u, checksum digits v, the checksum shift ls and the chain count p:

$$\begin{aligned} u &= \left\lceil \frac{8n}{w} \right\rceil = 64, \qquad v = \left\lceil \frac{\lfloor \log_2 \left(u(2^w - 1) \right) \rfloor + 1}{w} \right\rceil = \left\lceil \frac{\lfloor \log_2 960 \rfloor + 1}{4} \right\rceil = 3, \\ ls &= 16 - vw = 4, \qquad p = u + v = 67. \end{aligned} \tag{4.2}$$

Key generation draws 67 secrets $x_i = \mathrm{SHA256}(I \| u32(q) \| u16(i) \| \texttt{0xff} \| SEED)$ (RFC 8554 Appendix A), walks each chain to its end $z_i = c_i^{0 \to 15}(x_i)$, and publishes $K = \mathrm{SHA256}(I \| u32(q) \| \texttt{0x8080} \| z_0 \| \cdots \| z_{66})$. To sign a message M with randomizer C the signer hashes

$$Q = \mathrm{SHA256}(I \,\|\, u32(q) \,\|\, \texttt{0x8181} \,\|\, C \,\|\, M), \qquad S(Q) = \sum_{i=0}^{63} \Big(15 - a_i\Big) \in [0, 960], \tag{4.3}$$

where $a_0, \ldots, a_{63}$ are the base-16 digits of Q, most significant first, and the checksum digits a_{64}, a_{65}, a_{66} are the top three nibbles of the 16-bit word $S \ll ls$. Since $S \le 960 < 2^{12}$, these are exactly the three hexadecimal digits of S. The signature is $(C, y_0, \ldots, y_{66})$ with $y_i = c_i^{0 \to a_i}(x_i)$, which is $32 + 67 \cdot 32 = 2,176$ bytes. The verifier recomputes Q, continues every chain to its end, $z'_i = c_i^{a_i \to 15}(y_i)$, and accepts if and only if $\mathrm{SHA256}(I \| u32(q) \| \texttt{0x8080} \| z'_0 \| \cdots \| z'_{66}) = K$, compared in constant time.

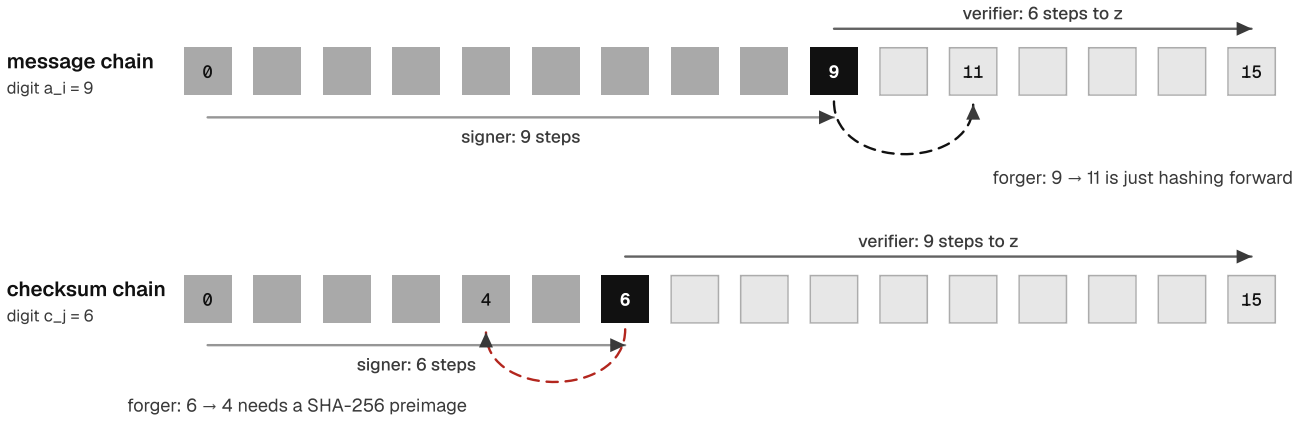


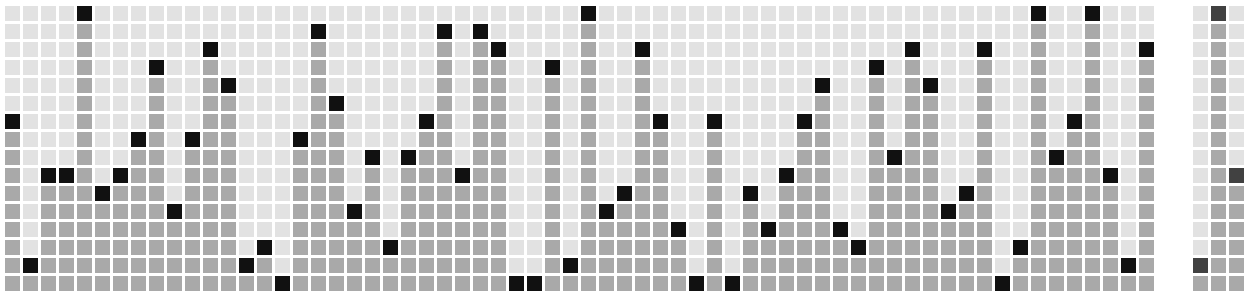
FIG. 4 **Why the checksum stops a forger.** Each of the 67 chains has 16 positions; the secret x_i sits at 0 and the public end z_i at 15. A signature reveals position a_i. Anyone can hash forward, so a forger can raise a message digit (top). But raising message digits lowers the checksum, so some checksum digit would have to fall (bottom): that is a step backwards, a preimage of SHA-256 (Lemma 4.1).

LEMMA 4.1 *(dominance).*

Let $a \in \{0, \ldots, 15\}^{67}$ be the digit vector (message digits and checksum digits) of a digest Q, and b that of a digest Q'. If $b_i \ge a_i$ for all 67 indices, then $b = a$, and therefore $Q' = Q$.

Proof. From $b_i \geq a_i$ on the 64 message digits, $S(Q') = \sum (15 - b_i) \leq \sum (15 - a_i) = S(Q)$. The checksum digits are the hexadecimal digits of S, and a three-digit number whose every digit is at least the corresponding digit of another is at least that number, so $S(Q') \geq S(Q)$. Hence $S(Q') = S(Q)$, i.e. $\sum_{i<64} (b_i - a_i) = 0$ with every term non-negative, so $b_i = a_i$ for every message digit, and the checksum digits agree because the checksums do. The 64 message digits are all 256 bits of the digest, so $Q' = Q$. ∎

A forger that holds one signature knows chain i at position a_i and can compute any later position by hashing forward. By Lemma 4.1 the only digests it can sign this way are those equal to Q: it must either find $(C', M') \neq (C, M)$ with the same 256-bit tweaked digest (a second-preimage search, success probability 2^{-256} per attempt classically, about 2^{128} Grover iterations), or step a chain backwards (a preimage of SHA-256). Katz gives the full reduction of LM-OTS's security to these properties of the hash in the random-oracle model [12]; the lemma is its combinatorial core and the reason the checksum exists. Fig. 5 computes a real signature over any text you type.



C (FIRST 8 BYTES)	Q (FIRST 8 BYTES)	CHECKSUM	VERIFIER STEPS
f491ea39a6f4da1a	9166f568c48db120	502 = 0x1f6	525 ≈ 81k CU

VERIFIED
yes: K' = K

FIG. 5 **A real LM-OTS signature, computed in your browser.** Type a message. The page computes $Q = H(I \| q \| \texttt{0x8181} \| C \| M)$ with the deterministic randomizer C, splits Q into 64 base-16 digits, appends the 3 checksum digits, signs with a demo key (derived from a public string, so it protects nothing) and verifies. Each column is one chain: the bright cell is the revealed value, filled cells below it the signer's hashes, hollow cells above it the verifier's.

The verifier's work is $T = \sum_{i<67} (15 - a_i) = S + \sum_{j<3} (15 - c_j(S))$ chain steps. For a uniformly random digest S is a sum of 64 independent uniform variables on $\{0, \ldots, 15\}$:

$$E[S] = 64 \cdot 7.5 = 480, \qquad \mathrm{Var}[S] = 64 \cdot \frac{16^2 - 1}{12} = 1360, \qquad \sigma_S \approx 36.9, \tag{4.4}$$

and the exact distribution of T (Fig. 6) follows by convolution. The worst case is the all-zero digest: $S = 960 = \texttt{0x3C0}$, checksum digits $(3, 12, 0)$, so $T = 960 + 12 + 3 + 15 = 990$. Each step hashes 55 bytes, which the `sol_sha256` syscall prices at $85 + \lfloor 55 / 2 \rfloor = 112$ CU; the measured cost of a whole LMS verification on SBF is $8,674 + 137\,T$ CU MEASURED.

LEMMA 4.3 *(the work is a multiple of 15).*

For every digest, $T \equiv 0 \pmod{15}$.

Proof. $T = S + 45 - (c_0 + c_1 + c_2)$ and $S = 256\,c_0 + 16\,c_1 + c_2$. Since $16 \equiv 1 \pmod{15}$, $S \equiv c_0 + c_1 + c_2$, so $T \equiv 45 \equiv 0 \pmod{15}$. The mean 505.9, median 510, 99th percentile 600 and maximum 990 are consistent with it; only the mean, an average, is not itself a multiple of 15. ∎

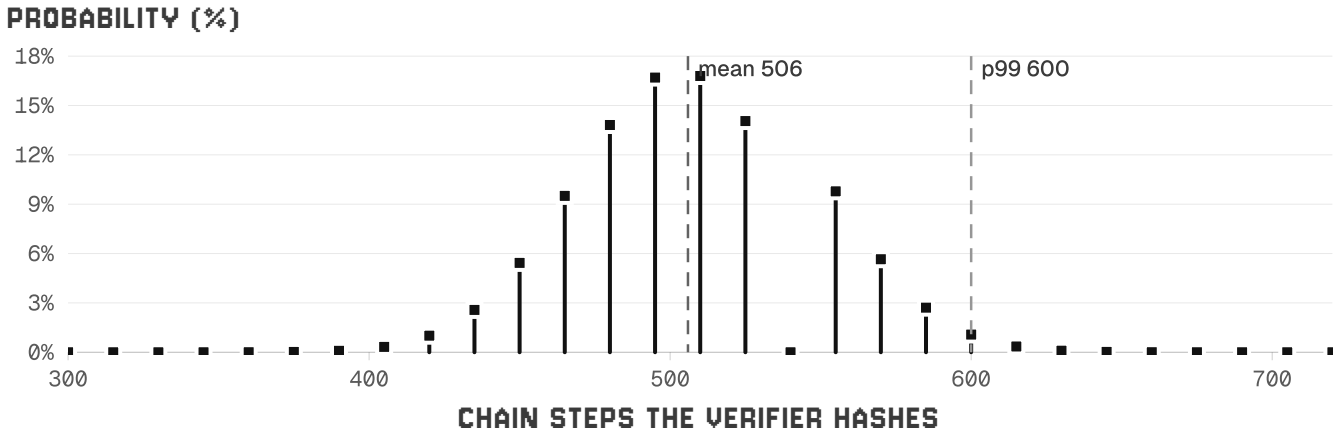


FIG. 6 **Verification work for a random digest (exact distribution).** $T = \sum_{i<67}(15 - a_i)$ chain steps, computed exactly by convolving 64 uniform digits and mapping each checksum to its three digits; T only takes multiples of 15 (Lemma 4.3). Mean 505.9, median 510, 99th percentile 600, maximum 990 (an all-zero digest). At the measured $8,674 + 137\,T$ CU of `lms_verify` that is about 78k CU on average, 91k at the 99th percentile and 144k at worst. Values outside 300-720 steps carry less than 10^{-6} of the mass and are cut from the plot.

4.2 The deterministic randomizer

RFC 8554 has the signer draw C at random. Its purpose is to make Q unpredictable until the message is fixed, so that an attacker who influences part of a signed message cannot prepare colliding messages in advance. Noised derives it instead, with HMAC [4] keyed by the key's secret seed, as FIPS 205 does for SLH-DSA's PRF_{msg} [3]:

$$C = \text{HMAC-SHA256}\left(SEED,\ \texttt{"phalanx/pq/lmots/C"} \,\|\, I \,\|\, u32(q) \,\|\, M\right). \tag{4.5}$$

PROPOSITION 4.2 *(idempotent signing).*

Under (4.5), signing the same message twice with the same key yields byte-identical signatures and reveals no chain value beyond the first signature. If HMAC-SHA256 is a pseudorandom function, C is as unpredictable to anyone without $SEED$ as a random draw.

Proof. C is a function of $(SEED, I, q, M)$ only, so Q, the digits and every y_i are too. The second claim is the PRF definition applied to an adversary that sees (I, q, M) but not the key. ∎

This matters operationally. A wallet that loses a record of a signature it already broadcast, a second device, or a restarted process may sign the same withdrawal again. With a random C that produces a second, different digest under the same key, which is exactly the case Fig. 8 shows to be dangerous; with (4.5) it reproduces the first signature. Every Noised signer must reproduce the known answer $SEED = \texttt{0x11}^{32}$, $I = \texttt{0x22}^{16}$, $q = 7$, $M = \texttt{"noised"}$, giving `C = efff9f33…9d79d3` (SPEC §3.10).

4.3 Why a second signature leaks

Suppose a key has signed k digests with digit vectors $a^{(1)}, \ldots, a^{(k)}$. The forger knows chain i from position $m_i = \min_j a_i^{(j)}$ on, and a fresh digest with digits b is forgeable exactly when $b_i \geq m_i$ for all 67 chains. The message digits of a random digest are independent and uniform; conditioned on $b_i \geq m_i$, each $15 - b_i$ is uniform on $\{0, \ldots, 15 - m_i\}$. So the success probability of one attempt is

$$\rho(m) = \prod_{i=0}^{63} \frac{16 - m_i}{16} \cdot \Pr\Big[c_j\left(S'\right) \geq m_{64+j},\ j = 0,1,2 \Big], \qquad S' = \sum_{i=0}^{63} U_i,\ U_i \sim \mathcal{U}\{0, \ldots, 15 - m_i\}, \tag{4.6}$$

which we evaluate exactly by convolving the 64 uniform distributions (all terms non-negative, so even probabilities near 2^{-256} keep full precision) and summing over the checksums whose digits clear m_{64}, m_{65}, m_{66}. The expected classical work is $1 / \rho$ digest evaluations, and Grover's search needs about $(\pi / 4)\rho^{-1/2}$ iterations. Fig. 8 shows the result: from 2^{256} at $k = 1$ to a feasible search at $k = 2$ for a large fraction of key pairs. The forger may choose the message freely (in a vault withdrawal: its own recipient, the whole balance, its own next key), and the randomizer gives it unlimited attempts, so ρ is the right measure. A one-time key must sign once; Noised enforces this on-chain for both schemes (§5, §10).

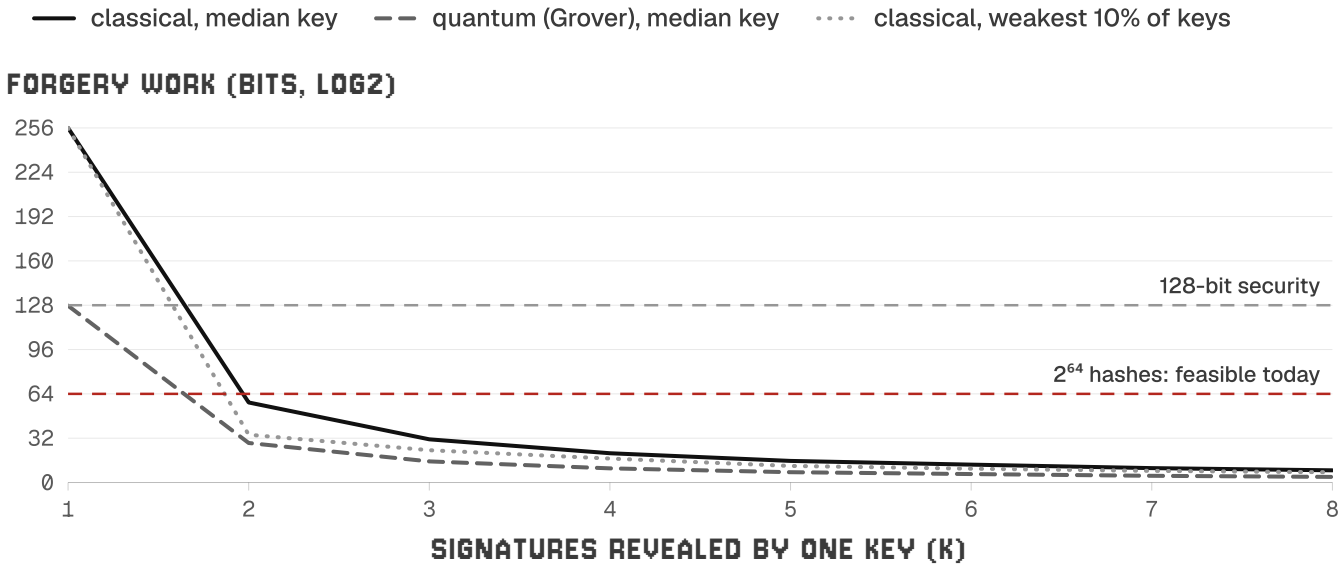


FIG. 8 **Forgery cost after k signatures by one LM-OTS key.** log2 of the expected number of digests a forger must try, per key, when it holds k signatures of independent random digests by the same key: exact per-key probabilities (equation 4.6), 64 random keys per k, deterministic seed. With one signature the cost is exactly 2^{256} (Lemma 4.1). With two it collapses to a median of $2^{57.8}$, and the weakest 10% of key pairs fall to $2^{34.5}$ or less: minutes of hashing on one ordinary computer. The dashed line applies Grover's square root. Hence one key, one signature, enforced on-chain (§10, §5).

4.4 LMS trees

An LMS key of height $h = 10$ authenticates $2^{10} = 1,024$ LM-OTS leaf keys under one 32-byte root [1, 13]. With K_q the LM-OTS public key of leaf q and node numbers r (root $r = 1$, leaves $r = 2^h + q$):

$$T[r] = \begin{cases} \mathrm{SHA256}\left(I \| u32(r) \| \text{0x8282} \| K_{r - 2^h}\right) & r \geq 2^h \\ \mathrm{SHA256}\left(I \| u32(r) \| \text{0x8383} \| T[2r] \| T[2r+1]\right) & 1 \leq r < 2^h \end{cases} \tag{4.7}$$

The public key is $u32(6)\|u32(3)\|I\|T[1]$ (56 bytes) and a signature is $u32(q)\|\sigma_{\mathrm{OTS}}\|u32(6)\|\mathrm{path}_0\|\cdots\|\mathrm{path}_9$, $4+4+2,176+4+320=2,508$ bytes. The verifier computes the candidate K_c from the one-time signature, hashes it into leaf $r=2^h+q$, and climbs: at each level it hashes its node with the next path element on the correct side and halves r, ten times, then compares with $T[1]$ in constant time (Fig. 7). Noised's verifier fixes every length and typecode and returns the leaf index read from the signature itself, so no caller can record one leaf while the signature used another. Key generation costs $1,024\times(67+67\cdot 15+1)+2,047=1,100,799$ SHA-256 calls (about 1.14 million compressions); a Ryzen 7 5800X does it in 62 ms natively **MEASURED**.

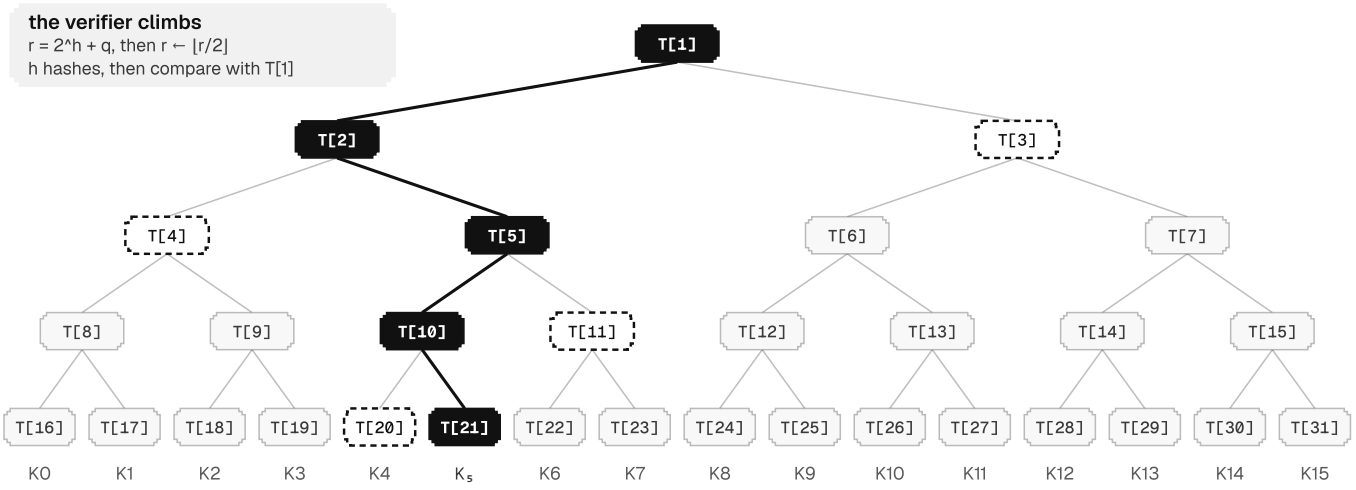


FIG. 7 **An LMS tree and one authentication path.** Drawn with height 4 (16 leaves) for legibility; a Noised identity has height 10 (1,024 leaves, a 10-node path). Leaf $q=5$ is node $T[2^h+q]=T[21]$. The signature carries the LM-OTS signature of that leaf and the siblings on its way up (outlined); the verifier recomputes the filled nodes and compares the result with the public root $T[1]$.

TABLE 3 **Parameters of the two schemes.**

QUANTITY	LM-OTS (VAULT KEYS)	LMS (CREATOR IDENTITIES)
RFC 8554 typecode	`LMOTS_SHA256_N32_W4 = 3`	`LMS_SHA256_M32_H10 = 6 (over type 3)`
Hash, output	SHA-256, n = 32	SHA-256, m = 32
Chains / height	p = 67 (64 + 3), 16 positions each	h = 10, 1,024 one-time leaves
Public key	I, q, K: 52 bytes (key id = SHA256(0x20 || u32(3) || I || u32(q) || K))	56 bytes (identity id = SHA256(0x21 || pk))
Signature	`2,176 bytes`	`2,508 bytes`
Verification	T chain steps + 2 hashes (mean T ≈ 506, max 990)	T + 13 hashes; 8,674 + 137 T CU measured
State	vault sequence number, on-chain	1,024-bit used-leaf bitmap, on-chain
Generic security	about 256 bits classical, 128 bits quantum (a Grover search over 256 bits)	

4.5 Standards status

NIST SP 800-208 [2] approves LMS (with HSS) and XMSS as stateful hash-based signature schemes, and both parameter sets Noised uses are among those it lists. Two of its conditions are worth stating precisely, because they bound what "standard" means here. First, the standard is explicit that stateful schemes are not suited to general use: a signer that ever uses a one-time key twice loses its security, so they are intended for settings where state can be managed reliably. Second, it requires key generation and signing to happen inside hardware cryptographic modules that do not export secret keying material. Noised clients generate keys in software (from a passkey secret or a recovery phrase) and sign in the browser or on an offline machine. Noised therefore uses *SP 800-208 parameter sets*; it is not an SP 800-208-conformant deployment, and we do not claim it is. Its answer to the state problem is different from the standard's: the state that matters is kept and enforced on-chain (the vault's sequence, the identity's bitmap), clients reserve every one-time key in a durable ledger before it signs, signatures are idempotent (Proposition 4.2), and every revealed signature can always land so that its key is consumed. FIPS 205 (SLH-DSA) is stateless but its smallest signatures, about 7.9 KB, do not fit one Solana transaction (§14).

05 Post-quantum creator identity, attestation and on-chain metadata

After an elliptic-curve break, any wallet can be impersonated. A launchpad whose notion of "who launched this" rests on a wallet signature then has no notion of origin at all. Protocol v2 gives every launch a post-quantum origin: the creator holds an LMS key (§4), the launch names it, and the launchpad refuses to make the launch live until that key has signed the launch's exact terms. Safety policy 3 extends the signature to the token's face: its name, symbol, description and icon are a record on-chain whose hash the identity signs. The verification, the one-time state and the record all live on-chain.

5.1 Keys

An identity is derived from a 64-byte $master$ secret, a cluster byte and an identity index:

$$\begin{aligned} path &= cluster \,\|\, u32_{le}(index) \\ SEED &= \mathrm{SHA256}(\texttt{"phalanx/pq/identity/v1/seed"} \,\|\, master \,\|\, path) \\ I &= \mathrm{SHA256}(\texttt{"phalanx/pq/identity/v1/id"} \,\|\, master \,\|\, path)[0..16) \\ pq_identity &= \mathrm{SHA256}(\texttt{0x21} \,\|\, u32(6) \,\|\, u32(3) \,\|\, I \,\|\, T[1]) \end{aligned} \tag{5.1}$$

The domain bytes are disjoint by construction: `0x20` names vault keys, `0x21` identities, the RFC's `0x80` - `0x83` occur only after I inside RFC hashes, and ASCII tags `phalanx/pq/…` start every client-side derivation (they keep the project's original name: they are fixed protocol constants, and renaming them would change every key). The cluster byte keeps a devnet identity from ever being a mainnet one, and every signer first checks the RPC's genesis hash. Two sources of $master$ are supported:

- **A dedicated recovery phrase**: 24 BIP-39 words (256 bits of entropy) [17], the BIP-39 seed being $master$. It is the same phrase that controls the user's vaults, with a different derivation path.
- **A passkey**, through the WebAuthn PRF extension (CTAP2 `hmac-secret`) [16], with user verification: the authenticator returns $prf = \mathrm{HMAC\text{-}SHA256}(CredRandom, \mathrm{SHA256}(\texttt{"WebAuthn PRF"} \| \texttt{0x00} \| salt))$, computed inside the authenticator from a per-credential secret, and the client takes $entropy = \mathrm{HKDF\text{-}SHA256}(prf, \texttt{"phalanx/pq/passkey/v1"}, \texttt{"…/bip39-entropy"}, 32)$ as the 24 words' entropy, so the derived $master$ can be exported as words for backup. It is a symmetric computation: a quantum computer gains only Grover's square root against it. The user trusts the authenticator, and for synced passkeys

the provider that syncs them; the PRF secret reaches roaming authenticators over classical channels; and the credential is scoped to the app's exact host (its relying-party identifier), so the same passkey does not work from another origin.

The SDK (0.4) ships both paths in the browser: key generation of the 1,024 leaves runs in time slices with progress (about 0.6 s in Node on a desktop CPU), and an authenticated cache of the tree lets an identity load in milliseconds afterwards.

5.2 Why the identity is never derived from a wallet

A tempting design derives the post-quantum secret from the user's existing wallet ("no new seed phrase"). Wallets do not reveal private keys, so such a derivation must hash a wallet *signature* on a fixed message. The argument against it is often stated as "a quantum computer recovers the wallet key and recomputes the signature". That is imprecise, and the precise version is both more subtle and more damning.

DEFINITION 5.1 *(wallet-derived secret).*

$s = F(\sigma^*)$ for a public function F, where $\sigma^* = \mathrm{Sign}_k(m^*)$ is the wallet's RFC 8032 Ed25519 signature on a fixed message m^* under secret key k with public key A.

PROPOSITION 5.2.

(i) Any party that obtains σ^* obtains s; a wallet returns σ^* to any application the user approves one message-signing request for, today, with no quantum computer. (ii) Shor's algorithm applied to A does not by itself yield s. (iii) Every step that binds the identity to a creator through an Ed25519 signature is forgeable by $\mathcal{A}_{\mathrm{q}}$.

Proof. (i) is the definition plus the wallet interface. For (ii), RFC 8032 §5.1.6 signs with $r = \mathrm{SHA512}(prefix \| m) \bmod \ell$, $R = [r]B$, $S = (r + \mathrm{SHA512}(R\|A\|m)\, a) \bmod \ell$, where a and $prefix$ are the two halves of $\mathrm{SHA512}(k)$. Shor returns $a \bmod \ell$ from A, which suffices to forge any *new* signature, but the deterministic σ^* also needs $prefix$, which is not a function of A: recovering it is a SHA-512 preimage search[3]. (iii) follows from (2.1): with a, every Ed25519 statement "wallet A vouches for identity X" can be produced for any X, including the attacker's. ∎

So the honest objection is not that Shor recomputes the identity. It is that a wallet-derived identity is a secret held by an oracle that answers on request (i), and that whatever ties it to its owner is classical (iii); a derivation that touches the scalar a directly, as some key-derivation schemes do, is broken outright. Noised's design therefore requires: (R1) the identity's secret is independent of every Ed25519 key and of everything a wallet can be asked to sign; (R2) the program verifies the hash-based signature itself before the launch can go live; (R3) one-time state lives on-chain; (R4) the signed message binds everything the launch is, its metadata included. The creator's wallet still pays for and signs `create_launch`, and the certificate records it, but as information only: it has no power, and after an elliptic-curve break it proves nothing.

5.3 The attestation

The identity signs a 189-byte message (version 2) that the program rebuilds from the `Launch` account and the signed terms:

$$m_{\text{att}} = \texttt{"PHX-ATTEST-V2"} \,\|\, \pi_{\text{lp}} \,\|\, L \,\|\, h_{\text{params}} \,\|\, \mathit{creator} \\ \|\, h_{\text{meta}} \,\|\, i64_{le}(t_{\text{nb}}) \,\|\, i64_{le}(t_{\text{na}}) \tag{5.2}$$

with $13 + 5 \cdot 32 + 2 \cdot 8 = 189$ bytes, where π_{lp} is the launchpad's program id, L the launch address, h_{params} the SHA-256 of every creator choice (including `pq_identity` itself), h_{meta} the hash of the token's metadata record (5.5), and $[t_{\text{nb}}, t_{\text{na}}]$ the window in which `launch` may run. The first version of this message bound the name, symbol and uri plus a SHA-256 of an off-chain document; that document's availability then depended on DNS, TLS or a pinning service, which is why the record moved on-chain.

`attest_launch` is permissionless and is relayed by a throwaway fee payer in a version-1 transaction [7] of a constant 3,005 bytes for every record **FINAL BUILD**. It derives the identity account from the public key it receives, verifies the LMS signature over m_{att}, reads the leaf q from the signature and sets bit $used_{\lfloor q\,/\,8 \rfloor} \leftarrow used_{\lfloor q\,/\,8 \rfloor} \vee 2^{\,q \bmod 8}$, refusing a bit already set (`PqLeafUsed`). Only then does it decide whether the signature also *attests*: the identity must be the one the launch names, the record `["metadata", L, h_meta]` must exist, the window must be valid, and the attestation slot must be free. If so it writes `["attestation", L]` with the full signature; otherwise the leaf is simply burned (event `PqLeafConsumed`). This ordering is deliberate: every valid signature at an unused leaf lands, whatever it signed, so a signature that was revealed but never used can always be landed to consume its leaf. Over all 1,024 leaves of one identity `attest_launch` costs 79,772 to 114,476 CU (mean 95,439); the worst possible digest is projected at 169,882 CU, inside the 300,000 the SDK requests **FINAL BUILD**.

A window is valid when it is between one minute and one hour long, has not ended, opens at most a week ahead, and (since audit round 3's F1) still has a minute left when the attestation lands:

$$60 \le t_{\text{na}} - t_{\text{nb}} \le 3{,}600, \qquad t_{\text{nb}} \le t_{\text{now}} + 604{,}800, \qquad \max(t_{\text{now}},\, t_{\text{nb}}) + 60 \le t_{\text{na}}\,. \tag{5.3}$$

The slot is free when no attestation was written yet, or, with F1, when the launch is still `Configured` and its attestation's window passed unused ($t_{\text{now}} > t_{\text{na}}$); a replacement overwrites every field of the old one. An unexpired attestation, and that of a launch already live, are never replaced. Before F1 a launch nobody opened in its window could never go live, and an attestation landed in its window's last second fixed a window nobody could use (round 3's finding R3-1).

Relaying. Every account of `attest_launch` except the payer is fixed by the signed data: the launch by the message, the identity by the key, the attestation by the launch, the record by the launch and the signed hash. The metadata review found that a relayer naming another record account could turn an attesting signature into a burn, again at every retry; now any other account fails the whole transaction (`ConstraintSeeds`), nothing is written and the leaf is not burned, so the same bytes are simply sent again. A relayer can therefore land a signature or hold it back, but not change what it does. The client checks the attestation after every landing and stops re-signing after two burn-only landings for one launch, since each new signature spends one of 1,024 leaves.

PROPOSITION 5.3 *(attested origin).*

Assume LMS is existentially unforgeable when each leaf signs at most one message, and SHA-256 is second-preimage resistant. Then no adversary, including $\mathcal{A}_{\text{q}}$, can make a launch live whose certificate names identity X unless X's holder signed that launch's m_{att}; no attestation by X can be moved to another launch, other parameters, another metadata record or another window; and the mint shows exactly the record X signed.

Proof. `launch` requires the account `["attestation", L]` with matching launch and identity, runs only inside its window, and loads only the record at `["metadata", L, h_meta]` for the attestation's h_{meta}. The attestation is written only by `attest_launch`, only after $\text{LMS.Verify}(pk_X, m_{\text{att}}, \sigma) = q$ with $\text{SHA256}(\texttt{0x21} \| pk_X)$ equal to

the launch's `pq_identity`, which h_{params} also covers. A valid σ on m_{att} is either one X produced or a forgery. The message contains L, h_{params}, h_{meta} and the window, so a signature for one tuple does not verify for another. The record at the signed address holds bytes that hash to h_{meta} (5.5), and `launch` re-reads the mint's metadata and requires exactly those bytes. The bitmap prevents a second landed signature per leaf; the deterministic randomizer and the client's leaf ledger prevent a second revealed one.

Under $\mathcal{A}_{\text{q}}$, an attacker can still create launches that *name* any identity, since `create_launch` is signed by an Ed25519 wallet; such a launch simply never goes live (the integration suite pins this: `a5_a_launch_naming_another_creators_identity_never_goes_live`). It can also launch under its *own* identity with any name and ticker. Origin is cryptographic, not reputational: users learn which identity a project uses the way they learn any public key.

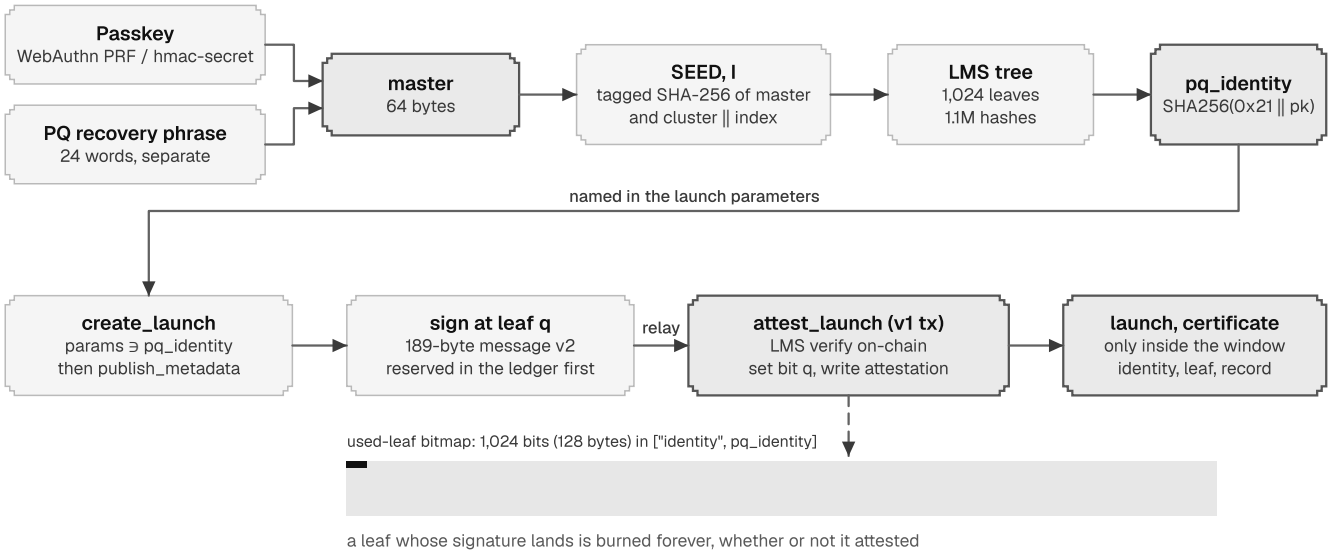


FIG. 9 **Identity lifecycle.** The identity's secret comes from a passkey or a dedicated phrase, never from a wallet. Its hash, `pq_identity`, is a launch parameter; the token's metadata record is published at the address of its own hash; one leaf signs the launch's attestation message, which names that hash; `attest_launch` verifies it on-chain and sets the leaf's bit in the identity's 1,024-bit map (three leaves used in the drawing); `launch` refuses to run without that attestation and gives the mint exactly the record.

5.4 Leaves: the on-chain bitmap and the client's ledger

The bitmap records signatures that *landed*. A signature that was revealed (simulated, dropped, sent in a transaction that failed) leaves its bit clear, yet a second message at that leaf would hand an observer two signatures of one one-time key. The client therefore keeps a durable leaf ledger on a store with an atomic insert: the leaf is reserved for its message before the signature exists, the signature is recorded before it is returned, and recorded signatures are re-broadcast, never re-signed. The first leaf a message tries is fixed by the message itself, $start = u16_{be}(\text{SHA256}(\texttt{"phalanx/pq/identity/leaf"} \| I \| m)[0..2]) \bmod 1024$, so a retry of the same message lands at the same leaf and two different messages rarely meet. One residual remains: two devices using one identity with two unshared ledgers put two messages on one leaf about once in 1,024 messages; the SDK's authenticated export and import of records narrows that window, and a single shared ledger closes it.

5.5 The token's metadata, on-chain

A token's name, symbol, uri mode and document (description, a small PNG icon, links) are one record, `TokenMeta`, written whole by the permissionless `publish_metadata` while the launch is `Configured`, at the address of its own content's hash:

$$h_{\text{meta}} = \text{SHA256}(\texttt{"PHX-META-V2"} \,\|\, \langle \mathit{name} \rangle \,\|\, \langle \mathit{symbol} \rangle \,\|\, \mathit{flags} \,\|\, \langle \mathit{uri} \rangle \,\|\, \langle \mathit{doc} \rangle) \tag{5.4}$$

where $\langle x \rangle = u32_{le}(\,|\,x\,|\,) \,\|\, x$ over UTF-8 bytes, and the record lives at the launchpad address $\text{PDA}(\texttt{"metadata"} \| L \| h_{\text{meta}})$. It is never modified or closed, and its size is exactly 91 bytes plus the four strings' lengths, from 93 to 3,405 bytes, with the document at most 3,072 bytes. Publishing is idempotent (an existing record already holds exactly these bytes) and takes over an address someone pre-funded. Anyone may publish other records under a launch, each at its own address; they are inert, because `attest_launch` and `launch` only ever use the hash the identity signed, and readers must resolve a record the same way: through `Attestation.metadata_hash` before launch or `Certificate.metadata` after, never by searching a launch's records. A typical record of 1,244 bytes (a 672-byte PNG icon) costs 6,969,760 lamports of rent at mainnet's rate[6], published in a 1,460-byte version-1 transaction `FINAL BUILD`.

> **PROPOSITION 5.4** *(content addressing).*
>
> Nobody can place bytes other than those the identity signed at a signed record's address, except by finding a SHA-256 second preimage of the record's encoding.

Proof. `publish_metadata` computes the address from the bytes it writes; the program rejects any other address for them. A record with different bytes at the signed address would need $\text{SHA256}(\texttt{"PHX-META-V2"} \| e') = h_{\text{meta}}$ for an encoding $e' \neq e$. The length prefixes make the encoding injective, so two different records never share an encoding. ∎

The signed `flags` byte chooses what `launch` writes into the mint's Token-2022 `uri`: nothing by default, the document inline as a `data:` uri (RFC 4648 base64 [25]), or an `https` link of the creator's choosing (Table 5). The mint holds the name and symbol itself, which every wallet and indexer shows. Icons in third-party wallets, however, come today from https-hosted JSON or from those wallets' own registries: no Solana token with a `data:` uri was found displaying anywhere, and the open-source DAS indexer stores uris in a `varchar(200)` column, so a longer uri may make it drop the token entirely [27]. The empty uri is the one value that indexer skips by design. Noised therefore defaults to it, the app shows the icon and description from the record (rendered only as an image of a PNG [26] of at most 64 × 64 pixels and as text, never fetched), and the inline mode stays out of the app until a wallet and indexer check on mainnet shows such tokens listed everywhere ordinary ones are. That is a limit of the ecosystem, not of the record: the bytes are on-chain and signed either way.

TABLE 5 What the signed uri mode writes into the mint, and who shows the icon.

MODE (FLAGS)	MINT URI	NAME AND SYMBOL	ICON AND DESCRIPTION	DEPENDS ON
none (0), the default	`""`	every wallet and indexer	the Noised app, the SDK, any reader of the record	nothing off-chain
INLINE_URI (1)	`data:application/json; base64,…` (≤ 4,125 B)	wallets that list the token; an indexer with a 200-character column may not	no wallet known to show it	nothing off-chain; not offered by the app
EXTERNAL_URI (2)	`https://…` (9 to 200 B, printable ASCII)	every wallet and indexer	wallets that fetch the link	DNS, TLS and a server; the content can change; not offered by the app

`launch` takes no arguments: it loads the record the attestation names, passes its name, symbol and derived uri to DBC, then re-reads the mint's metadata and requires exactly those values and no additional field (`MetadataMismatch`, which an honest DBC never triggers; it guards against an upgrade that altered the strings). The certificate sets bit 2048 (`METADATA_ONCHAIN`; all twelve checks are 4,095) and records the record's address.

TABLE 4 Proofs of origin compared.

	WALLET-DERIVED PQ IDENTITY	SIGNATURE IN OFF-CHAIN METADATA	NOISED IDENTITY
Secret comes from	A signature the wallet produces on request	Any key, stored with the token's metadata	Passkey PRF or a dedicated 24-word phrase; never a wallet
Who else can get it	Every app the user approves one signing request for	Not secret: it is a published claim	No one without the authenticator or phrase
Checked by	Client software	Client software, if anyone	The launchpad program, before the launch can go live
After an elliptic-curve break	Binding to the creator forgeable (Prop. 5.2 iii)	Proves nothing on-chain; the token's keys and pool stay classical	Unaffected: hash-based check on-chain; the wallet field has no power
One-time state	Client-side; a lost or forked state reuses a leaf	Not applicable	On-chain bitmap plus a durable client ledger; idempotent re-signing
What it binds	Whatever the client signs	A document, available while hosted	Launch, all parameters, the metadata record's every byte, window
Where the proof lives	Off-chain	IPFS or another store	On-chain, forever: attestation, record and certificate

IN ONE LINE

A launch on Noised goes live only after a hash-based key, which no quantum computer can derive from public data, signs its exact terms and the hash of its on-chain metadata, and the chain itself checks that signature. Table 4 is the comparison in full.

06 The fair-open batch

On a bonding curve the price rises with every purchase, so whoever buys first buys cheapest. On most launchpads that race is won by bots, bundlers and the creator itself, often inside the token's creation transaction. Noised replaces the race with a batch: during a commit window anyone deposits wSOL; one swap then buys for everyone at once; tokens and unspent wSOL are shared in proportion to what each committed.

6.1 Model

Let commits $c_1, \ldots, c_n > 0$ total $C = \sum_i c_i$, and let K be the batch's total cap (the migration threshold when the creator sets none; otherwise at least a tenth of it). `execute_batch` swaps $A = \min(C, K)$ in partial-fill mode with minimum output zero, measures from escrow balance differences the quote actually spent $S \le A$ and the tokens received $T > 0$, and each commit is later paid

$$tokens_i = \left\lfloor \frac{T\,c_i}{C} \right\rfloor, \qquad refund_i = \left\lfloor \frac{(C-S)\,c_i}{C} \right\rfloor, \tag{6.1}$$

computed in 128-bit integers. A batch that nobody executes in its window, or that cannot execute (the pool already traded, or nothing could be bought), *expires*, and every commit is refunded in full.

PROPOSITION 6.1 *(payouts are bounded).*

$\sum_i tokens_i \le T$ and $\sum_i refund_i \le C - S$; the escrows keep less than n base units and n lamports of rounding dust[4].

Proof. $\sum_i \lfloor Tc_i \,/\, C \rfloor \le \sum_i Tc_i \,/\, C = T$, and each floor loses less than 1. The same holds for the refunds with $C - S \ge 0$, which the program checks ($S \le A \le C$). ∎

PROPOSITION 6.2 *(one price).*

Up to rounding, every committer pays the same average price $S \,/\, T$ per token: $(c_i - refund_i) \,/\, tokens_i \approx \left(c_i - (C-S)c_i \,/\, C\right) \,/\, (Tc_i \,/\, C) = S \,/\, T$.

PROPOSITION 6.3 *(no gain from splitting, timing or order).*

Splitting a commit $c = c' + c''$ across wallets never increases what it receives: $\lfloor Tc' \,/\, C \rfloor + \lfloor Tc'' \,/\, C \rfloor \le \lfloor Tc \,/\, C \rfloor$, and likewise for refunds. The time and order of commits do not appear in (6.1).

Proof. $\lfloor x \rfloor + \lfloor y \rfloor \le \lfloor x + y \rfloor$ for all real x, y. Only c_i, C, S and T enter the formula. The property is pinned in the host tests (`splitting_across_wallets_gives_no_advantage`) and in LiteSVM against mainnet DBC: a creator whose commit inside the launch transaction is 80% of the total receives exactly 80% of the batch. ∎

Capital still counts: a participant who commits more gets proportionally more, which is the intended meaning of fair. A per-wallet cap `batch_wallet_cap` exists, at least $K \,/\, 200$ when set, but anyone can split across wallets, so it limits only naive concentration (§12).

Payout owners. A participant's first commit fixes where its payouts go: the tokens to `token_owner` and the unspent wSOL to `refund_owner`, each the committer or a Noised PQ vault of the paid mint, so the batch can pay straight into post-quantum custody and no key, Ed25519 or other, can redirect it afterwards. Claims are exempt from the Launch Guard, so a vault named by several committers could otherwise gather guard-exempt shares in one account (the PQ v2 review's proof of concept reached 62 times the max-wallet cap in one vault); `["payout-vault", L, vault]` therefore adds up every commit naming a vault and holds the total to one wallet's cap. The residual (round 3, R3-2): anyone may fill a vault's quota by naming it, which only donates that commit's tokens to the vault.

6.2 Oversubscription

Round 1 of the review found that a first-come-first-served cap let the creator fill the batch in the launch transaction itself (finding L1). The cap now never refuses a commit; it bounds what the swap spends. With $C > K$,

$$\frac{c_i - refund_i}{c_i} \;\approx\; \frac{S}{C} \;\le\; \frac{K}{C}, \qquad \frac{refund_i}{c_i} \;\approx\; 1 - \frac{S}{C}, \tag{6.2}$$

the same fractions for every commit (Fig. 10). The cap floors of protocol v2 ($K \geq threshold / 10$, a non-zero wallet cap at least $K / 200$) keep a creator from shrinking the fair open to dust while every other check passes.

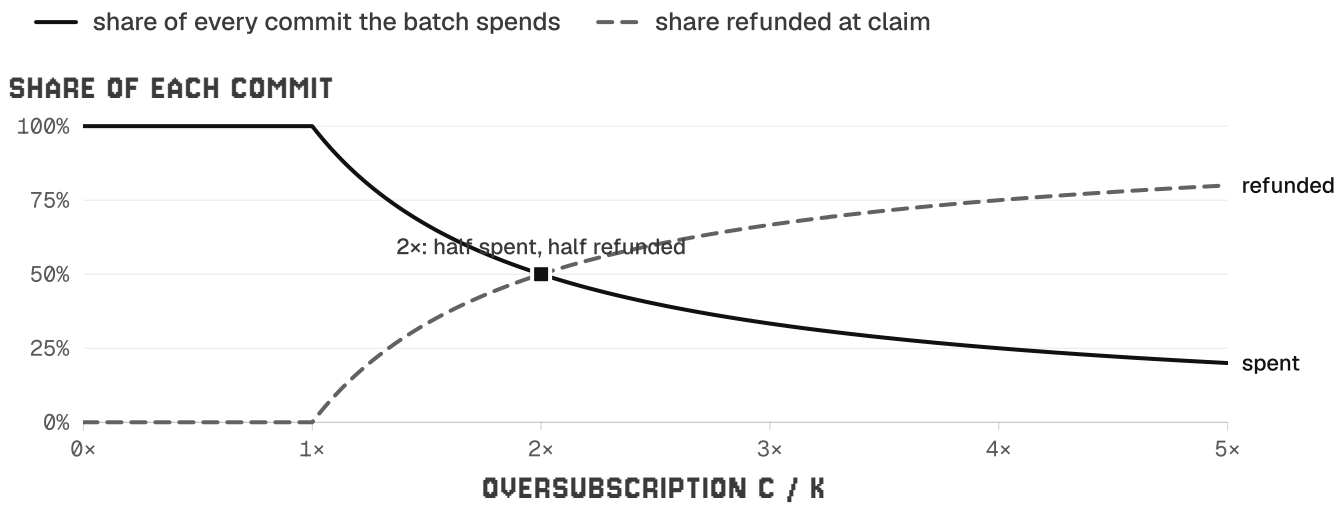


FIG. 10 **Pro-rata fill against oversubscription.** With commits totalling C against a cap K, every commit has the same fraction $\min(1, K / C)$ spent and the rest refunded, whatever its size, wallet or arrival time. First come, first served would instead fill the cap with the earliest commits and refuse the rest.

6.3 The price the batch pays

Meteora DBC prices with concentrated-liquidity segments: on a segment with liquidity L, moving the square-root price from s_a to s_b absorbs $\Delta q = L(s_b - s_a)$ of quote and releases $\Delta x = L(s_b - s_a) / (s_a s_b)$ of base (in DBC's Q64.64 units the quote carries a factor 2^{-128}).

> **LEMMA 6.4** *(the batch pays the geometric mean).*
>
> A purchase that moves the price from $p_a = s_a^2$ to $p_b = s_b^2$ within one segment pays an average price of $\sqrt{p_a p_b}$.

Proof. $\Delta q / \Delta x = L(s_b - s_a) \cdot s_a s_b / \Big(L(s_b - s_a)\Big) = s_a s_b = \sqrt{p_a p_b}$. ∎

Fig. 11 applies the lemma to the Fair preset: the batch turns a race from the first lamport's price to the last one's into a single price between them. Because the batch is the pool's first swap, that price starts from the curve's opening price. The hook makes that so (§8), and the launchpad checks it independently: `execute_batch` refuses to run when DBC's `has_swap` flag is set (`PoolAlreadyTraded`), so a batch whose pool was ever traded first can only expire and refund. This is why the swap may safely use a minimum output of zero.

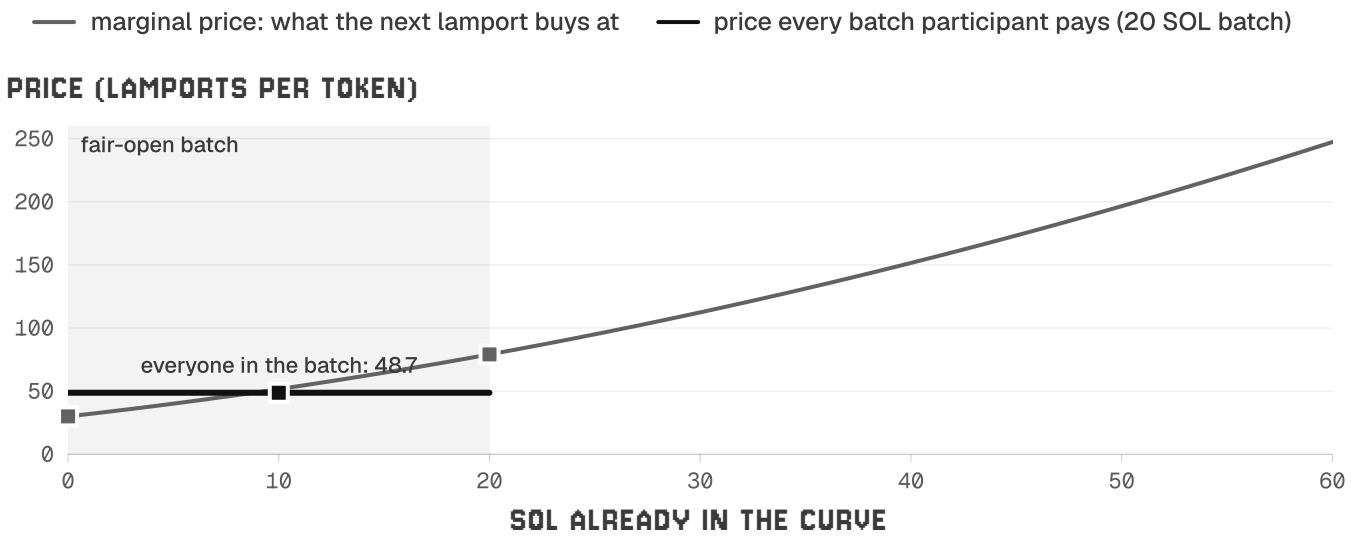


FIG. 11 **One price for the whole batch, on the Fair preset's real curve.** The SDK's Fair preset (start 0.00000003 SOL, graduation 0.0000004 SOL per token, 85 SOL threshold), recomputed with the SDK's exact curve arithmetic. Without a batch, the first buyer pays 30.0 lamports per token and whoever buys the 20th SOL pays 79.1: a race that bots and bundlers win. The batch spends the same 20 SOL in one swap and every participant pays the geometric mean, 48.7 lamports (Lemma 6.4). Fees are left out; they scale every price by the same factor.

6.4 The snipe shield

From the launch until the batch executes, or its execution window ends, nobody can buy (Fig. 12). Holding the shield for the whole window would delay public trading by up to ten minutes; instead `execute_batch` ends it immediately through the hook's `end_snipe_shield`, which only the launchpad's hook-authority PDA may call and which can only move the end earlier:

$$t_{\text{shield}} \leftarrow \min(t_{\text{shield}}, t_{\text{now}}) \quad \text{at the batch.} \tag{6.3}$$

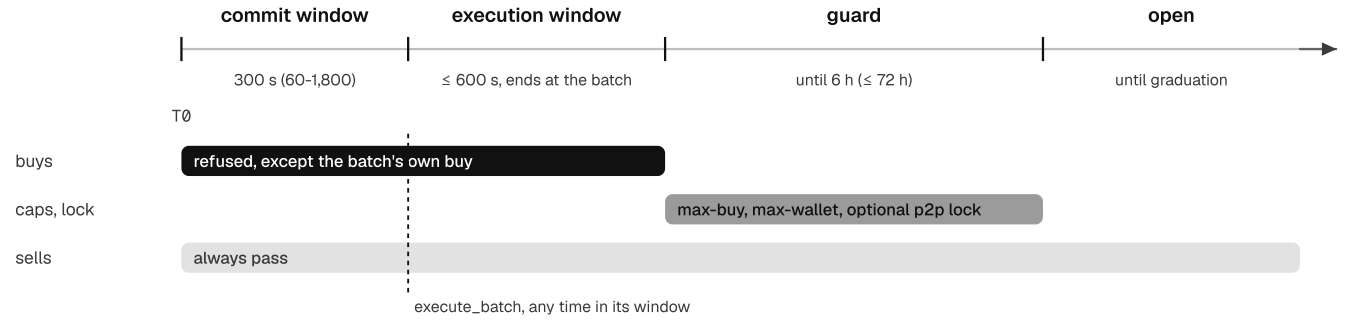


FIG. 12 **Snipe shield and guard timeline (Fair preset).** Segment widths are not to scale. Until the batch executes, or the execution window ends, the hook refuses every buy except the one inside a top-level `execute_batch`; the batch ends the shield at once, so a long window never delays trading. Max-buy, max-wallet and the optional wallet-to-wallet lock then apply until the guard ends. Sells into the pool pass at every moment.

07 Contention and liveness

Safety does not depend on anyone acting; liveness does. A batch executes only if someone sends `execute_batch` inside its window, and commits land only if they are included. Solana schedules transactions per block under compute limits, which gives a well-funded adversary a way to keep honest transactions out: write-lock the launch's accounts and fill their budget.

7.1 A cost model for write-lock contention

Agave 4.3.0 caps the compute units charged to any one writable account per block at $B = 24,000,000$ (with a 60M-CU block limit) [21]. A transaction can request at most $C_{\max} = 1,400,000$ CU and pays $f_0 = 5,000$ lamports per signature plus its compute-unit price π (micro-lamports per CU) times the CU it requests. Leaders order by price, so to exclude honest transactions paying π_h the attacker fills B in every block of a window of W seconds at price $\pi_h + 1$. With slot time $\tau \approx 0.4$ s:

$$\mathrm{Cost}(W, \pi_h) \;=\; \frac{W}{\tau}\left(\left\lceil \frac{B}{C_{\max}} \right\rceil f_0 \;+\; \frac{B\,(\pi_h + 1)}{10^6}\right) \text{ lamports}, \qquad \left\lceil \frac{B}{C_{\max}} \right\rceil = 18\,. \tag{7.1}$$

An honest commit, by contrast, pays $f_0 + c\,\pi_h \,/\, 10^6$ with a tight compute limit $c \approx 50,000$ CU (a commit measured 43,977 CU **V2 BUILD**): per block, the attacker's marginal cost is about $B \,/\, c = 480$ times the honest one. Fig. 13 plots (7.1) **MODEL**.

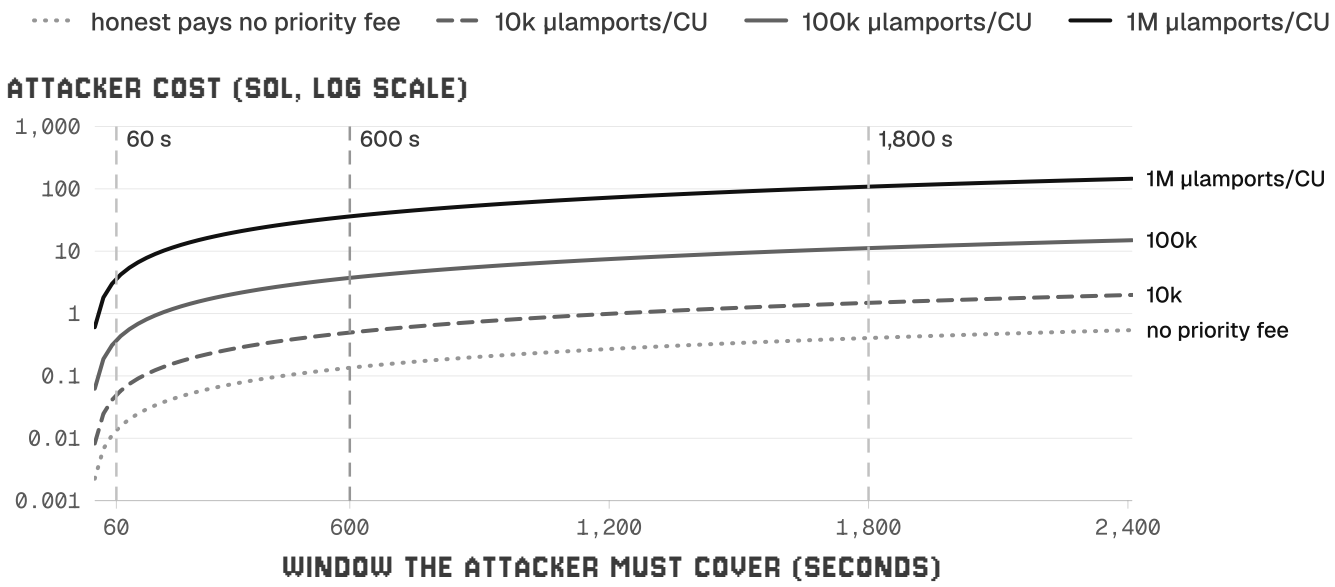


FIG. 13 **What it costs to keep honest transactions out of a launch's accounts.** Attacker SOL to fill the Launch account's per-block write budget (24M CU, 18 transactions per block) for a whole window, at a compute-unit price one micro-lamport above the honest transactions' (equation 7.1, 400 ms slots[5]). Vertical lines: the shortest commit window (60 s), the execution window (600 s) and the longest commit window (1,800 s). If nobody pays a priority fee, blocking a one-minute window costs about 0.014 SOL; if honest commits pay 1 lamport per CU, blocking the execution window costs about 36 SOL. A blocked batch refunds in full (§7).

7.2 Windows chosen from the model

- **Commit window, at least 60 s** (`MIN_OPEN_DELAY`): a window of seconds could be blocked for a fraction of a cent; a minute costs about 0.014 SOL even when nobody pays a priority fee, and the cost grows linearly with length and with honest priority.
- **Execution window, 600 s** (`EXECUTION_GRACE_SECS`, raised from 30 s in round 2): forcing a batch to expire costs ten minutes of contention, 0.14 SOL at zero priority and about 36 SOL against keepers paying 1 lamport per CU. Because the batch ends the shield (6.3), the long window costs honest users nothing when a keeper is prompt.
- **Launch window, 60 s to 1 h, chosen and signed by the creator**: every `launch` writes the `Launch` account, so the same contention can keep it out of a window while someone pays for it. The creator picks a longer window where contention is expected. A window that passes unused costs one leaf: since round 3's F1 the identity signs a new opening time, which replaces the expired attestation; before F1 such a launch could never go live.
- **Keepers**: `launch` (inside its window), `execute_batch`, `claim`, `claim_fees` and `distribute` are permissionless. The app, any participant and independent keepers can send them; `execute_batch` must be a top-level instruction and fits a legacy transaction.

HONEST SCOPE

The model raises the price of exclusion linearly; it does not remove it. It also assumes leaders follow fee priority: leaders and the ordering gateways in front of most of them (about 62% of leader slots) can delay or drop transactions for free. Noised's answer is that every timed step fails safe. A batch that does not execute expires and `claim`, which reads no DBC account, refunds every commit in full; a launch window that passes unused is signed again; a vault withdrawal or an attestation has no expiry of its own and is simply re-broadcast. Censorship can deny or delay a fair open; it cannot take funds or change what anyone signed.

08 Launch Guard and the transfer hook

Token-2022 calls a mint's transfer-hook program on every transfer [20]. Meteora DBC lets a pool's mint carry a hook and revokes it, program and authority, in the swap that completes the curve, so a hook exists only during the bonding-curve phase [18]. Noised's hook, Launch Guard, uses that phase for the snipe shield and bounded anti-sniping rules, and is built so that it can never become a honeypot.

8.1 Decision order

A transfer is a *buy* when its source is the pool's base vault, whatever its destination, and a *sell* when its destination is the base vault and its source is any other account. Fig. 14 gives the order in which `rules::evaluate` decides, after `execute` has checked that the policy belongs to the mint (step 1). The function is pure, total and allocation-free.

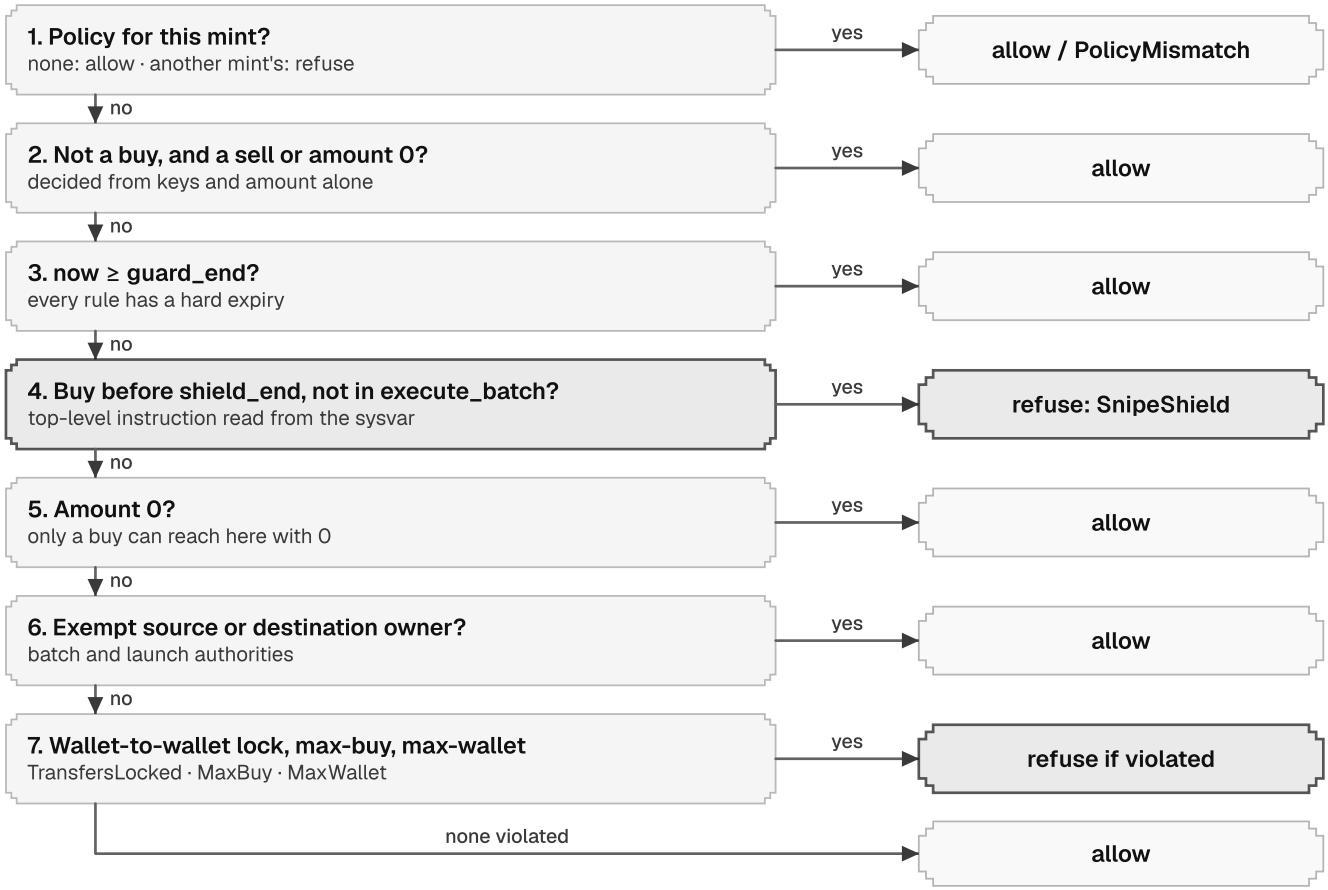


FIG. 14 **Launch Guard's decision order.** The order is the argument: sells (2) and expiry (3) are decided before any rule can refuse, so no rule can ever block a sell or outlive the guard; the shield (4) is checked before the zero-amount (5) and exemption (6) passes, so neither can open it. Audit round 3's F2 splits step 6: an exempt destination passes, then a receiving account without `ImmutableOwner` is refused (`MutableOwner`), then an exempt source passes.

THEOREM 8.1 *(no honeypot).*

For every policy and every transfer: a sell is allowed; any transfer at or after t_{guard} is allowed; a mint without a Noised policy is never restricted. Moreover $t_{\text{guard}} - t_{\text{launch}} \leq 72$ h for every policy the hook accepts, and the hook ceases to exist when the curve completes.

Proof. In `evaluate`, the sell test (step 2) and the expiry test (step 3) return before the first statement that can refuse; step 1 allows mints without a policy. `create_policy` accepts only a policy signed by the launchpad's hook-authority PDA whose guard is at most $72 \cdot 3600$ s and whose shield is at most 3,600 s, independently of the launchpad's own (stricter) validation. Revocation at completion is DBC 0.2.1 behaviour (FACTS #25). The first two claims are also property tests over 200,000 random transfers each in the host suite, and all three are LiteSVM tests against mainnet Token-2022. ■

One caveat is structural, not a rule: Token-2022 fails a transfer whose hook accounts are missing, so a client must pass the four accounts (meta list, policy, instructions sysvar, hook program). They depend only on the mint, the meta list is created in the same transaction as the mint, and it can never change, so any client can always derive them. A sell can be made to fail only by the seller's own client.

8.2 Recognizing the batch buy

During the shield the hook must pass exactly one buy. It reads the instructions sysvar: the transaction's *currently executing top-level instruction* must belong to the launchpad and start with `execute_batch`'s 8-byte discriminator. This is sound because the sysvar lists only top-level instructions with the program each one really invokes, the runtime keeps its current index at the top-level instruction during CPIs, and the sysvar accessors refuse any account that is not the real sysvar. A program that CPIs into the launchpad appears as itself, so a CPI'd `execute_batch` is refused. The shield is checked before the zero-amount and exemption passes: the review found that a buy into an account owned by an exempt PDA (finding L2), or a buy whose output rounds to zero (round 1b), still moved DBC's price and volatility accumulator, and so the fee the batch paid. The independent first-swap check of §6 backs the hook up.

8.3 Bounds

TABLE 6 **Launch Guard bounds, enforced twice.**

RULE	LAUNCHPAD POLICY (CREATE_LAUNCH)	HOOK (CREATE_POLICY)
Snipe shield	commit window 60-1,800 s + execution window 600 s; ended early by the batch	≤ 3,600 s
Guard (caps, lock)	from the shield's end, at most 72 h after launch	≤ 72 h
Max wallet	off, or 10-10,000 bps of the initial supply	≥ supply / 1,000
Max buy	off, or 1-10,000 bps	≥ supply / 10,000
Wallet-to-wallet lock	optional, never applies to buys or sells	during the guard only
Trading fee	floor base fee + largest dynamic fee ≤ 10%	not a hook rule

Caps are per transfer and per token account, so Sybil wallets get around them; they slow naive concentration during the first hours and are documented as such. Max wallet also applies to accounts many people pay into: a holder of about the cap can fill the shield's one escrow for the mint, or top up someone's associated token account, and further wraps or payouts into it fail until the guard ends (round 3, R3-5; griefing only, at most 72 h).

Owners that cannot change. The hook sees transfers, not ownership. Round 3 showed that a buyer could buy into a hand-made token account without Token-2022's `ImmutableOwner` extension and then hand the account to another wallet with `SetAuthority`, which moves no tokens, so the wallet-to-wallet lock was bypassed and one owner could gather many accounts each near the cap (R3-4). Fix F2 adds one refusal to `evaluate`: while the guard is on, after an exempt destination is allowed and before an exempt source is, a receiving account without `ImmutableOwner` is refused (`MutableOwner`, hook error 6010). Associated token accounts always carry the extension, so buys, claims, vault `release` payouts and shield wraps are unaffected, and sells never reach the rule. With F2 the lock stops token transfers and consolidation by owner reassignment during the guard; it cannot stop several people sharing control of one wallet (a multisig or a smart wallet), which needs no transfer at all. Launch Guard's own work is small: one policy read, two fixed-size account reads, the sysvar read only for buys during the shield, and a loop over at most four exempt keys. Measured, the hook's `execute` costs 2,262-4,404 CU and a whole hooked Token-2022 transfer 15,878-23,378 CU **DOCS**.

09 Liquidity and fees

The launchpad fixes every Meteora DBC configuration field a creator could use against holders and lets the creator choose only the curve, fees within bounds, the batch and guard parameters, and up to four fee recipients. Among the fixed values: quote mint wSOL; the mint's metadata immutable; 100% of migrated liquidity permanently locked under the launch-authority PDA; no migration fee; no creator allocation or vesting (creators buy in the batch like everyone else); unsold base tokens burned at migration. After `launch`, the program checks the fresh mint's post-conditions (no mint, freeze or metadata authority; exactly the expected extensions; the hook is Launch Guard) and aborts if any fails.

9.1 Bonding curve and graduation

A curve is up to eight segments (s_k, L_k) of constant liquidity between square-root prices. Integrating the segment formulas of §6 from the start price, after q lamports have entered the curve its square-root price and the base it has released are

$$s(q) = s_0 + \frac{q}{L}, \qquad x(q) = L\left(\frac{1}{s_0} - \frac{1}{s(q)}\right), \qquad p(q) = s(q)^2 \quad \text{(one segment, DBC units)}. \tag{9.1}$$

When the curve holds the migration threshold, anyone can call DBC's permissionless migration to DAMM v2. Since `partner_permanent_locked_liquidity_percentage` is 100, the whole position is permanently locked, and DAMM v2's permanent lock is irreversible; the launch authority keeps only the right to claim the position's fees. Fig. 15 draws the three presets.

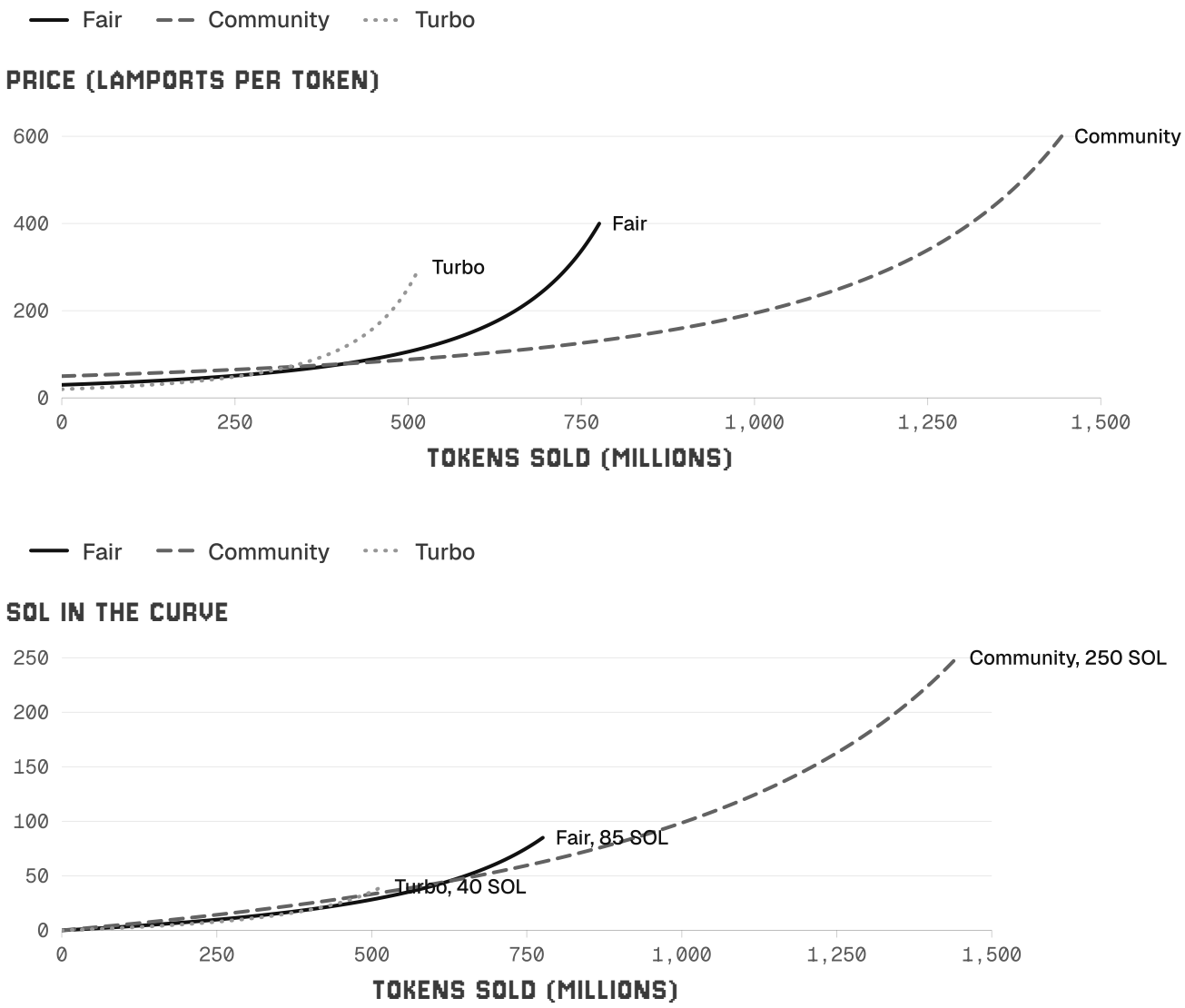


FIG. 15 **The three presets' bonding curves, from the SDK's exact parameters.** Price (left) and SOL held by the curve (right) against tokens sold, up to each preset's graduation threshold. Each preset is one constant-liquidity segment from its start price to its graduation price, sized so the curve holds exactly the threshold there; at graduation DBC moves the reserve and the matching tokens into a DAMM v2 pool whose liquidity is then locked forever. The price grows as the square of $s = s_0 + q\,/\,L$, i.e. quadratically in the SOL raised.

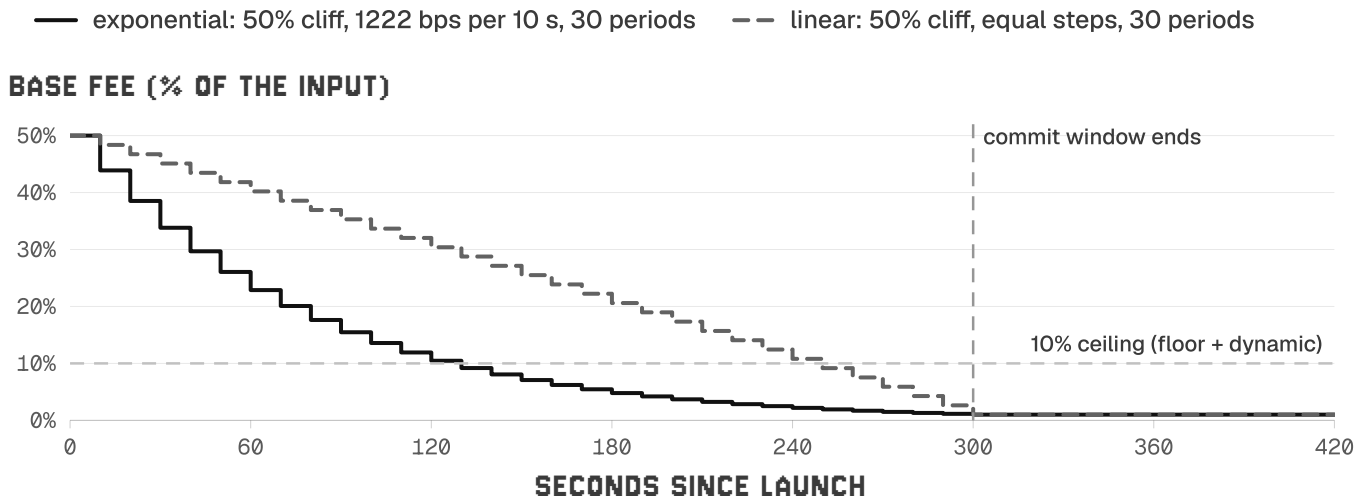
9.2 Fees and their routing

Every DBC trade pays a base fee plus an optional dynamic (volatility) fee, both in the quote token. DBC keeps 20% as its protocol share; the rest accrues to the partner, which is the launch-authority PDA (the creator's own trading-fee share is fixed at 0). The policy requires any decaying base-fee schedule to end within the commit window, so the batch pays the floor (Fig. 16). Anyone may call `claim_fees` and then `distribute`, which splits the launch authority's whole wSOL balance W:

$$platform = \left\lfloor \frac{W\, b_{\mathrm{p}}}{10^4} \right\rfloor, \qquad pay_j = \left\lfloor \frac{(W - platform)\, b_j}{10^4} \right\rfloor (j < r), \qquad pay_r = W - platform - \sum_{j<r} pay_j, \tag{9.2}$$

with the platform share $b_{\mathrm{p}} \le 5,000$ bps snapshotted from the immutable `Protocol` account at creation, and recipient shares b_j summing to 10,000, fixed at `create_launch` forever. The split always sums to W. After

graduation, `claim_lp_fees` collects the locked position's fees: wSOL is distributed the same way, and launch tokens are burned, never sold. Recipients may be wSOL PQ vaults, and the treasury must be one (§10), so fees accrue under post-quantum custody. Because `distribute` pays $\mathrm{ATA}(\mathit{recipient}, \mathrm{wSOL})$ and the legacy `create_launch` has no room to inspect recipient accounts, a PQ vault of *another* mint named as a recipient would strand its whole share; the SDK and the app refuse such recipients before the creator signs (round 3, F3, a client check). The graduated pool's fee tier is one of DAMM v2's fixed options, 0.25%, 0.3%, 1%, 2%, 4%, 6%; DAMM v2's operators can later raise a pool's base fee to at most 10%, which §12 lists among the trusted parties.



`first_factor` = periods N, `second_factor` = seconds per period f, `third_factor` = reduction per period.

FIG. 16 **Base-fee schedules the policy accepts for a 300-second commit window.** Two schedules, computed with the launchpad's bit-exact mirror of DBC's fee scheduler (the exponential one with DBC's Q64.64 square-and-multiply). The policy requires the whole schedule to end within the commit window ($N \cdot f \le \delta$), so the batch, which executes after it, pays the floor (1.00% here) plus at most the dynamic fee, and floor plus maximum dynamic fee may not exceed 10%. A decaying fee is DBC's usual defence against snipers; under Noised no buy is possible before the batch anyway, so the presets use a flat 1-1.5% fee.

10 The PQ vault (v4)

The vault gives post-quantum custody inside a chain that is not post-quantum. Tokens in a vault move only as payouts authorized by an LM-OTS signature (§4); no Ed25519 key has any authority over them. The design problem is not the signature scheme but one-time-ness: a revealed signature that cannot execute tempts its owner to sign again, and a second signature is a forgery waiting to happen (Fig. 8). Version 4, after review round 1, removes every way for a revealed signature to fail.

10.1 Protocol

A vault is the PDA `["vault", mint, genesis_key_id]`; its token account is the deposit address and never moves (Fig. 17). A withdrawal signs the 185-byte message

$$m_k = \texttt{"PHX-PQV4"} \,\|\, \texttt{0x01} \,\|\, \pi_{\mathrm{vault}} \,\|\, V \,\|\, \mathit{mint} \,\|\, u64_{le}(k) \,\|\, \mathit{dest} \,\|\, u64_{le}(\mathit{max}) \,\|\, \mathit{id}(K_{k+1}) \qquad (10.1)$$

with the current key K_k, where k is the vault's sequence. The program checks the signature, that the supplied next key hashes to the signed id and differs from the current key, and (protocol v2) that the keys' leaf indices follow the sequence: $q(K_0) = 0$ at open and $q(K_{k+1}) = k + 1$ at withdrawal (`InvalidKeyIndex`). It then reserves

$$amount = \begin{cases} 0 & dest = V \\ \min\left(max,\ balance - R \right) & \text{otherwise} \end{cases}, \qquad R \leftarrow R + amount, \tag{10.2}$$

creates `["payout", V, u64le(k)]`, sets $k \leftarrow k + 1$ and installs K_{k+1}. Keys are derived from a separate 24-word phrase per (cluster, mint, account, sequence), so keys never repeat; the randomizer is deterministic (4.5). The signed amount is a maximum, so a withdrawal never fails for insufficient funds.

PROPOSITION 10.1 *(invariants).*

(i) $R \leq balance$ before and after every instruction. (ii) No signature executes twice, and no retired key can act on the vault again. (iii) A correct signature for the current sequence always executes: `withdraw` reads no account that a third party controls.

Proof. (i) `withdraw` adds at most $balance - R$; delivery subtracts exactly what it transfers; deposits only raise the balance, and only the vault PDA, which never does so, could move tokens out other than by delivery. (ii) The message contains k, and the instruction that accepts it increments k and replaces the key with one that must differ; the genesis key id is the PDA seed, so `open_vault` for it fails forever. (iii) `withdraw` touches the submitter's own signer, the mint (fixed and, being eligible, never closable), the vault (this program's), the vault's token account (read-only, only its balance read; a frozen account still reads) and a fresh payout PDA only this program can create. No recipient account is read or written. ■

Version 3 paid the recipient inside `withdraw`; a recipient could then make a revealed signature fail forever by reassigning its token account's owner, requiring memos, or refusing non-confidential credits (finding V1, high). In v4 such a recipient delays only its own payout: `release` pays the canonical ATA (creating it, sending a memo first when required), and `release_to`, signed by the recipient, pays any token account it names (while a Launch Guard is on, only one whose owner cannot change: round 3's F2). `open_vault` refuses mints that leave an Ed25519 path to the deposit: a permanent delegate, a close authority, live transfer-fee authorities, and non-transferable mints.

Never signing twice, in practice. Proposition 10.1 (ii) concerns signatures that land; the dangerous case is a second *revealed* one. The SDK therefore requires a durable vault-key ledger for every signature: the key is reserved for its withdrawal on a store with an atomic insert (and, in a browser, a cross-tab lock) before any signature exists, the signature is recorded and verified before it is returned, the vault's history is searched for a withdrawal the key already signed (and the signer refuses while that search is incomplete), and the fallback builder simulates only signatures the ledger holds. Because a signed recipient can never change, the signer also requires the user's separate confirmations of the recipient check (cautions about the recipient, about the mint, and an off-curve recipient), runs that check again right before signing, and refuses if anything new appeared. Two devices holding one phrase with two unshared ledgers could still each sign once; they share a ledger, or move authenticated records before the second device signs.

10.2 Cost

`withdraw` carries 2,312 bytes of instruction data, so it travels in a version-1 transaction (4,096 bytes, live since 2026-09-15), paid by a throwaway fee payer; no user wallet ever signs a version-1 transaction. Table 7 collects the published measurements.

TABLE 7 **Measured vault costs (LiteSVM, mainnet token programs, 26 runs, 2026-10-08).** Ranges reflect the per-message verification work and address bump searches.

TRANSACTION	COMPUTE (CU)	V1 SIZE (BYTES)	NOTES
withdraw	79k-106k	2,693	a legacy encoding would be 2,777 bytes, over the 1,232 limit
release, SPL Token, ATA created	34k-58k	522	also fits a legacy transaction
release, Token-2022 + Launch Guard	63k-97k	654	forwards the 4 hook accounts
withdraw + release, SPL Token	121k-172k	2,845	0.50 MB of loaded account data
withdraw + release, Token-2022 + hook	145k-190k	2,977	1.95 MB of loaded account data
Payout account	-	-	122 bytes; 1,270,000 lamports of rent, refunded to its payer at delivery

The SHA-256 syscall model predicts about 58k CU of hashing on average and 112k at worst for one LM-OTS verification, consistent with the measured whole-instruction range DOCS.

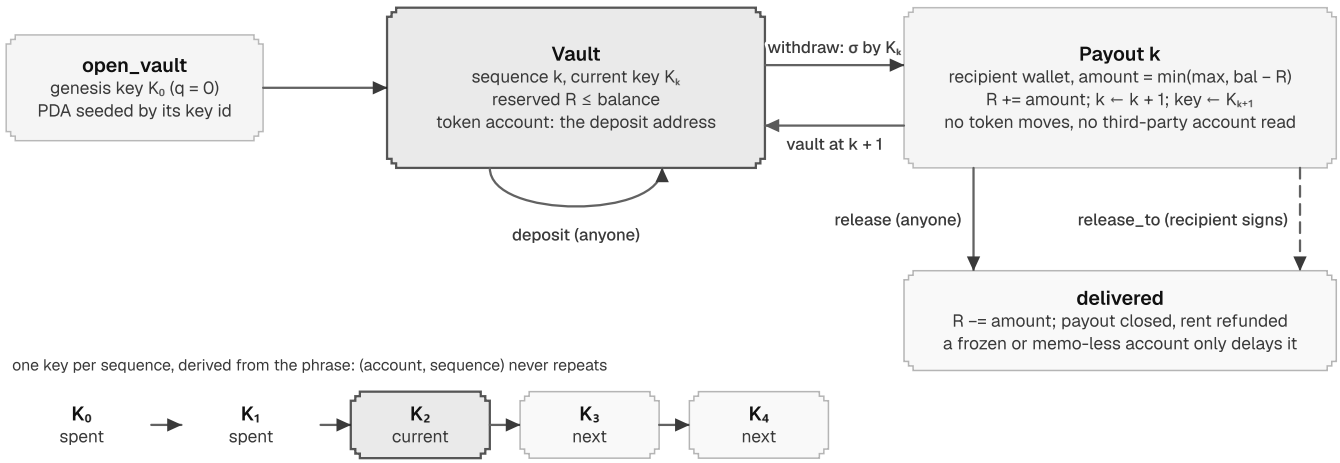


FIG. 17 **The vault's state machine.** The vault address and its token account never change. Each withdrawal is one signature by the current one-time key over the vault's current sequence; it reserves a payout, increments the sequence and installs the signed next key in place. Delivery is a separate, permissionless step that only the payout's recipient can delay, and only for itself.

11 Private balances

AMMs cannot trade Token-2022 mints with confidential transfers, so privacy is a separate, reversible layer next to the tradeable token. The shield wraps any eligible token 1:1 into a zToken, a Token-2022 mint with the Confidential Transfer extension (no auditor key, no mint authority except the shield PDA, no freeze authority). Unwrapping burns

zTokens and returns the base token. After every instruction the program asserts $\mathrm{supply}(z) \leq \mathrm{balance}(\mathit{escrow})$, minting the escrow's measured balance difference so that transfer fees cannot break the invariant (equality cannot be enforced: anyone can donate to the escrow, and a one-unit donation must not brick the shield). A zToken copies its base mint's uri only if it is at most 200 bytes with no invisible character, so a Noised token's empty uri is copied as is. Confidential operations themselves are plain Token-2022 instructions.

TABLE 8 What private balances hide, and what they do not.

HIDDEN	NOT HIDDEN
Confidential balances of zToken accounts	Who owns which accounts, and who transacts with whom (the transaction graph)
Amounts of confidential transfers	Wrap and unwrap amounts (they are ordinary token transfers), and their timing
	Everything recorded today from a quantum attacker later: ElGamal over Ristretto is not post-quantum (harvest now, decrypt later)
	Availability: Token-2022 is upgradeable, and the ZK ElGamal proof program has a feature-gate kill switch (used in 2025-26)

The shield is therefore described as private balances, not anonymity, and it is opt-in. Its LiteSVM suite (13 tests) covers wrap and unwrap round trips, donations and direct burns that try to break the supply invariant, every refused base-mint extension, Launch Guard accounts forwarded through wraps, and a confidential transfer between accounts.

12 Security evaluation

12.1 Invariants

Table 9 is the audit checklist: each line is a property the programs enforce in every reachable state, and each is pinned by tests that replay attacks against the mainnet binaries of DBC, DAMM v2, Token-2022 and SPL Token.

TABLE 9 Invariants (SPEC §7, protocol v2 and safety policy 3).

NAME	PROPERTY
Mint	Every certified mint has no mint, freeze or metadata authority and exactly the extensions MetadataPointer, TokenMetadata and TransferHook (Launch Guard) until graduation; then the hook is revoked.
Config	Every launchpad-created DBC configuration has exactly the fixed safe values.
Sells	No sell, and no zero-amount transfer that is not a buy, is ever refused; no rule applies after the guard; mints without a policy are never restricted.
Batch	Σ commits = total; quote spent ≤ min(total, cap); each commit is paid once; payouts never exceed tokens out and unspent quote; expired batches refund 100%.
Fees	distribute pays at most the balance; the platform share uses the snapshotted bps; recipients never change after create_launch.
Funds	No instruction moves launchpad-PDA funds except the batch buy, batch claims, fee distribution and base-token burns.
No admin	Protocol cannot be modified; no pause; programs are finalized after audit.
Vault	Tokens leave a vault only through release / release_to of a payout created by a valid LM-OTS signature of the current key over the exact message; the sequence increments and the key rotates on every withdrawal.
Shield	zToken supply never exceeds the escrow balance, after every instruction (donations can only add surplus).
Executable	A valid vault signature always executes: withdraw reads no account a third party controls.
Reserve	reserved ≤ vault token balance, before and after every vault instruction.
One buy	Before the shield ends, the only buy is the batch buy, inside a top-level execute_batch.
First swap	The batch is the pool's first swap, or it refunds.
Origin (v2)	launch runs only with an attestation by the identity the launch names, over its exact parameters, metadata record and window, and only inside that window.
Leaves (v2)	Each LMS leaf of an identity lands at most once, ever; every valid signature at an unused leaf lands.
Shield end (v2)	Only the launchpad's hook-authority PDA can end a shield, and only earlier, never later.
Key index (v2)	A vault's key at sequence k has leaf index k.
Payout owners (v2)	claim pays only the payout owners the first commit fixed; a vault named by several commits receives at most one wallet's cap.
Metadata (v3)	A TokenMeta record is never modified or closed and sits at the hash of its own content; launch gives the mint exactly the record its attestation names (name, symbol, derived uri, no other field).
Relay (v3)	Every account of attest_launch except the payer is fixed by the signed data; any other record account fails the transaction and burns nothing.

12.2 Adversarial review

Round 1 (2026-10-08) ran seven finders in parallel, each with one lens (accounts and PDAs; CPI and Meteora integration; Token-2022 and the hook; economics; post-quantum crypto and the vault state machine; liveness and DoS; policy completeness), all reading the specification, the code and the Meteora, Anchor and Token-2022

sources, and writing proof-of-concept tests against mainnet binaries in scratch copies. Each finding then went to one to three skeptics instructed to answer "not real" unless the scenario held. Fixes were checked by two further adversarial reviewers; the launchpad reviewer found one more path through the shield (a buy whose output rounds to zero), closed in round 1b in two layers. Table 10 lists round 1's issues.

TABLE 10 **Round 1 and 1b: confirmed issues.**

ID	SEVERITY	FINDING	STATUS
V1	High	A recipient could block a signed withdrawal and freeze the vault (v3 paid inside withdraw).	Fixed: vault v4
L1	High	The creator could fill the batch cap in the launch transaction (first come, first served).	Fixed: pro-rata fill
L2	High	A buy into an exempt account passed the shield and raised the batch's dynamic fee; residual: zero-output buys.	Fixed: sysvar check, first-swap rule
V3	Medium	A fixed transfer fee was charged on the whole balance at every withdrawal (successor vaults).	Fixed: rotate in place
V4	Medium	A mint with a close authority could be re-created with a permanent delegate.	Fixed: refused at open
V2	Low	A withdrawal to the vault itself or its successor could be signed but never executed.	Fixed: v4
L3	Low	A reassigned wSOL ATA blocked its owner's claim.	Fixed, with a trade-off
V5	Low	SOL or other mints sent to the vault PDA itself are unrecoverable.	Documented limitation
C1	Low	Re-signing the same withdrawal drew a fresh randomizer (two signatures).	Fixed in the crate and the SDK
C2	Low	No client checked the recipient before signing.	Fixed: recipient check and confirmation

Every later change was reviewed the same way (Table 11, Fig. 18). Round 2 added a lens on the documents' own claims and found that write-lock contention made the 10-second commit and 30-second execution windows cheap to block (§7); its program fixes are protocol v2. A spec review issued 17 binding amendments to the post-quantum design before any code was written (among them: the identity binds the metadata, every revealed signature must be landable, and "NIST approved" and "audited" were not facts), and an adversarial review of the result found an unburnable leaf that let a third party forge a second signature, fixed by verifying first and always burning the leaf (§5). The on-chain metadata review found that a relayer could turn an attesting signature into a burn by naming another record account, fixed by deriving that account from the signed hash. Round 3, on the final programs, confirmed five findings, recorded 92 areas checked and found sound, and led to two program fixes, both merged: F1 (an expired, unused attestation can be replaced, and a window needs 60 s left when it lands) and F2 (guarded transfers only into token accounts with a fixed owner).

TABLE 11 **Every internal review.** Severities as confirmed; round 1b's other results (3 weak tests, 1 documentation claim, 1 size limit) were not severity-rated.

REVIEW	DATE	REVIEWERS	HIGH	MEDIUM	LOW	INFO	OUTCOME
Round 1	2026-10-08	7 finders, 45 skeptics	3	2	5	0	every program fix with regression tests; V5 a documented limitation
Round 1b	2026-10-08	2 adversarial reviewers, 2 test agents	1	0	0	0	closed in two layers (hook, first-swap rule); weak tests rewritten
Clients review	2026-10-08	1 reviewer	0	3	5	1	fixed in SDK 0.2 and 0.3 and in the app
Round 2	2026-10-08	8 finders, 34 skeptics	1	4	11	0	program fixes in protocol v2; contention mitigated, a residual
PQ v2 adversarial	2026-10-09	1 reviewer, proof of concept	0	2	3	2	all fixed
SDK 0.2 and 0.3	2026-10-09	3 reviewers	0	5	9	6	all fixed
On-chain metadata	2026-10-09	1 reviewer, proof of concept	0	1	0	1	fixed: any other record account fails
Round 3	2026-10-10	6 lenses, 2 skeptics per finding	0	1	2	2	F1, F2 fixed and merged; F3 client check; F4 documented

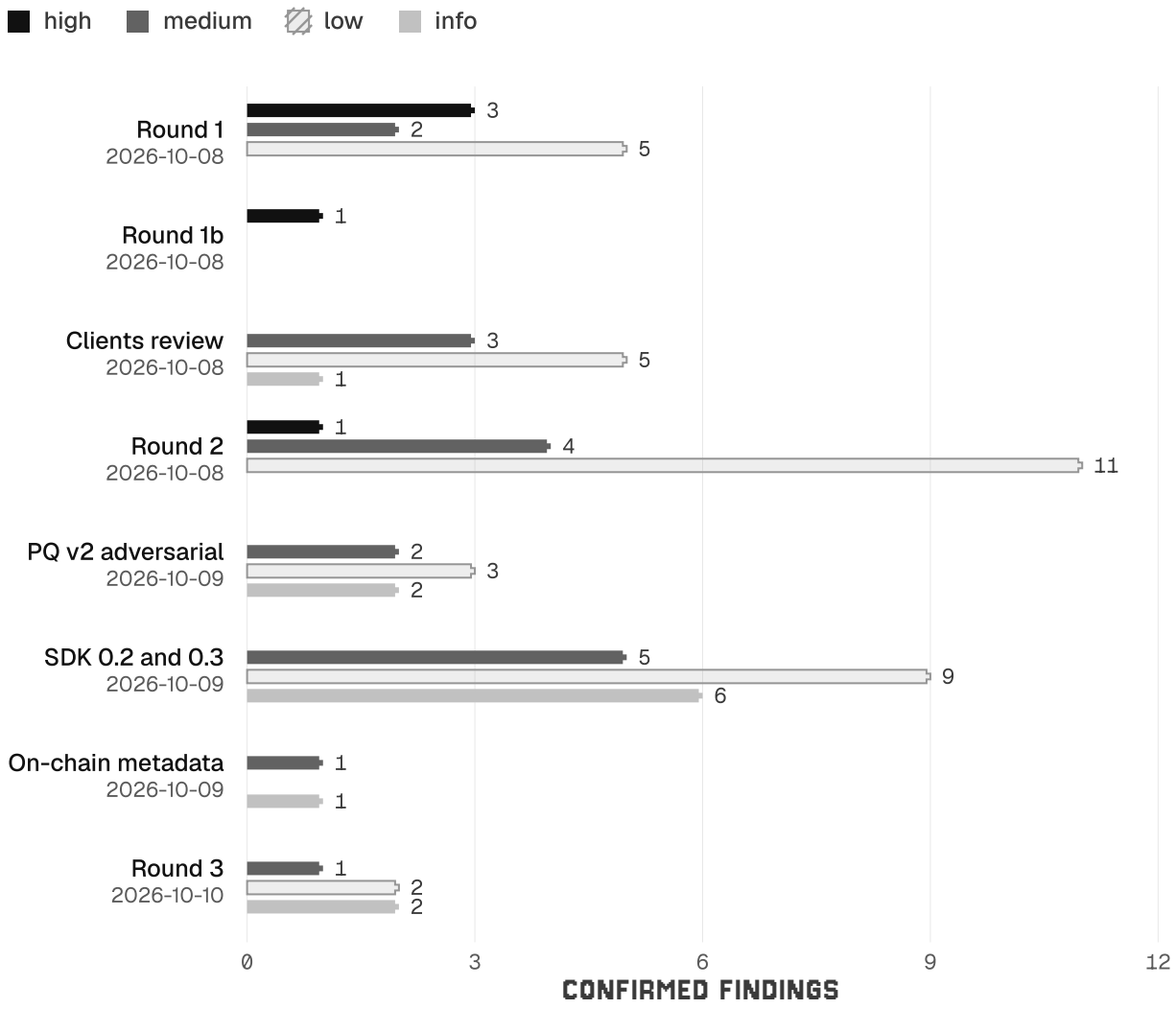


FIG. 18 **Confirmed findings of every internal review, by severity.** 5 high, 18 medium, 35 low and 12 informational findings over eight reviews, each confirmed by skeptics or a proof of concept. Every high and medium finding is fixed or, for write-lock contention, made linearly more expensive; round 3's two program fixes are merged. No finding was confirmed against the shield's own code or the LM-OTS and LMS verification. These were internal reviews by AI agents [24].

12.3 Testing

TABLE 12 **Test results.** Round 1b as published; v2 from the integration build's logs; final from the final verification of the programs with on-chain metadata (before round 3's fixes) and SDK 0.4.0.

SUITE	ROUND 1B, 2026-10-08	V2 BUILD, 2026-10-09	FINAL BUILD, 2026-10-09
Host unit and property tests (programs)	78	106	151
noised-lmots (RFC 8554 vectors, HMAC, randomizer, LMS)	12	34	35
Vault, hook and launchpad integration (LiteSVM, mainnet token programs)	64	84	98
Launchpad end to end (LiteSVM, mainnet DBC and DAMM v2)	17	46	59
Shield (LiteSVM)	13	13	17
Mutation testing: broken program copies caught by the integration suite	59 / 59	102 / 102	148 / 148
Mutation testing: launchpad mutants only the LiteSVM suite reaches	-	8 / 8	23 / 23
Mutation testing: LMS implementation	-	31 / 31	unchanged crate
SDK (byte parity with Rust-produced vectors)	225	467	545
Mutation testing: the SDK's new code	-	-	128 / 131

The LM-OTS and LMS code reproduces RFC 8554 Appendix F byte for byte, including the full height-10 tree of Test Case 2 rebuilt from its 1,024 one-time keys, and its HMAC matches RFC 4231 [5]; three independent Python re-derivations match every committed vector, the metadata records included. The final integration mutants are 38 vault, 36 hook and 74 launchpad mutants; two more LiteSVM mutants survive by design, because they change only the `MetadataMismatch` comparison, which an honest DBC can never reach (host tests pin it directly). The SDK's three surviving mutants are equivalent re-checks kept as defence in depth. Round 3's F1 and F2 came with their own tests (the audit's proofs of concept as LiteSVM tests, plus integration tests of the hook) and kill every mutant of their new rules: 43 hook and 74 launchpad integration mutants, 17 LiteSVM and 8 host mutants.

12.4 Residual risks and trusted parties

- **Write-lock contention** can keep commits, the batch or `launch` out of their windows while someone pays for it; the batch then refunds in full, and an unused launch window is signed again (F1; §7).
- **The first transaction in a window wins**, and whoever relays an attestation decides when it lands; nobody can change what a signed transaction does.
- **Two devices, two ledgers.** A phrase or identity used on two devices that do not share their one-time-key records can sign two messages with one key; the SDK's authenticated export and import narrow the window, and a shared ledger closes it.
- Claims and payouts follow a reassigned SPL Token ATA (so that nobody but the holder can block them); a wallet tricked into signing an owner change loses later refunds to the new owner.
- Anyone may fill a PQ vault's batch quota by naming it in a commit (the tokens go to the vault, the unspent wSOL back to the griefer's own refund owner); during the guard anyone may fill a shared account (the shield's escrow, shared by every holder, or someone's ATA) to max wallet; and the wallet-to-wallet lock stops transfers, not changes of control without one: a shared wallet, a PDA or smart-wallet owner changing hands, a PQ vault's key rotation, or a delegate approval.
- Launch Guard trusts the launchpad's `execute_batch` to make exactly one swap; any change to it must keep that property.
- A hook upgraded in place with a different account list would break every existing mint's transfers, so it must never be done.
- The batch is pro rata: capital still counts, and per-wallet caps are Sybil-splittable.

TABLE 13 Who a launch still trusts.

PARTY	WHAT IT CAN DO
Meteora upgrade multisig (4-of-7, no timelock)	Replace DBC and DAMM v2, including behaviour the shield relies on; a batch that then cannot execute refunds.
DAMM v2 admins and operators	Disable a graduated pool (locked liquidity stuck), raise its base fee up to 10%, close migration configs.
Token-2022 upgrade multisig (4-of-6); the ATA program	Replace the token program governing every Noised token, zToken and Token-2022 vault; create every account Noised pays.
Validators (feature gates), consensus	Change the runtime; Ed25519 vote keys are not post-quantum.
Leaders and ordering gateways	Reorder, delay or drop transactions and decide who lands first in a window; cannot change signed content.
Wallets; passkey authenticators and sync providers	Show what the user signs and hold classical keys; a synced passkey's provider can move the identity's PRF secret.
Noised's upgrade key, until finalization	Replace any of the four programs. Removed by finalizing after audit.
Frontend, DNS, RPC, wallet	Show false information or ask for a harmful signature; verify with the SDK against several RPCs.

BOTTOM LINE

The evidence is local: LiteSVM against mainnet program dumps and host tests, reviewed by AI agents in eight reviews. The programs stay upgradeable until they are finalized (§14).

12.5 The QSL-1 standard

The guarantees above are written down as a named, versioned token standard, QSL-1 (“Quantum Security Layer token, version 1”), so that a wallet, an explorer or an indexer can recognise them without trusting Noised’s word. QSL-1 does not change Token-2022: a QSL-1 token is an ordinary Token-2022 mint with a fixed profile (no mint or freeze authority, immutable metadata, exactly three extensions), layered the way ERC-4626 is layered on ERC-20. The launchpad, which refuses to launch anything else, records what it verified in the certificate, one immutable account per mint; a verifier re-checks every condition from live accounts and never trusts the certificate’s flags alone.

- **Level 1, No-rug.** Nobody can mint more, freeze holders, change the metadata, withdraw the graduated liquidity, block sales or redirect fees; 24 conditions over the mint, the DBC configuration, the pool, the hook policy and, after migration, the locked position, ending with finalized programs.
- **Level 2, Quantum-attested.** Level 1, and the launch went live only because the creator’s LMS identity signed its parameters, its on-chain metadata record and its window (§5); the 189-byte message is rebuilt from account state and the signature re-verified.

There is no third level. A token whose creator added launch rules or automation is *programmable*: a tag shown next to its level (“QSL-1 L2 · programmable — read its rules”) with plain-English warnings, never a level and never “safer”. A rule set can be signed, bound and enforced exactly as written and still let only the creator’s chosen wallets buy for a while: binding proves who set the rules, not that they are good for holders.

A verifier awards no level while the programs can still be upgraded or the endpoint is not mainnet-beta: until finalization every token is at most a *candidate*, which is what the app shows on devnet. The SDK’s `checkQsl1` implements the algorithm, and the app’s badge uses only the standard’s own wording, which forbids “quantum-proof”: a QSL-1 token is held in ordinary wallets, and level 2 is post-quantum launch authorization and metadata integrity inside a chain that is not post-quantum.

13 Performance

All measurements are LiteSVM 0.17 runs with the mainnet feature set, SBPF v3 builds of the Noised programs and the mainnet binaries of DBC 0.2.1, DAMM v2, Token-2022 v11 and SPL Token. Fig. 19 covers every instruction a launch, a trade, a vault or a shield uses; Fig. 20 the transaction sizes. Values from the final build `FINAL BUILD` are the programs with on-chain metadata after their review fix (2026-10-09), as reported by that stage; values from the v2 build `V2 BUILD` include the post-quantum attestation and the round-2 changes (the batch now also ends the shield, 13,653 CU, and creates the base escrow) and are the latest exact measurements of instructions the metadata change did not touch (their final-build ranges are in each bar's note); round-1b values `DOCS` are as published.

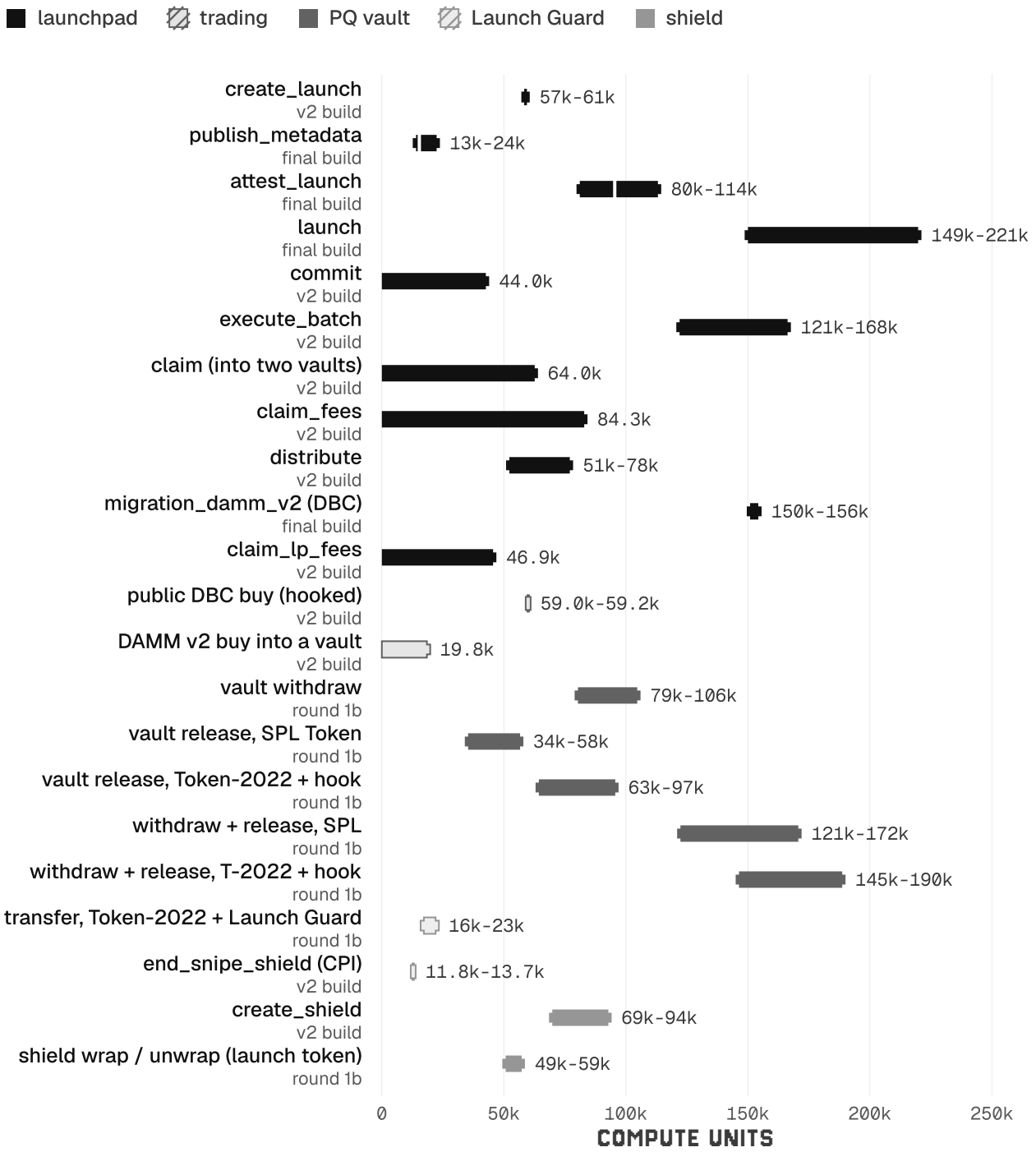


FIG. 19 **Compute per instruction, measured.** Ranges are minimum to maximum over all runs (random addresses change bump searches; signature verification depends on the digest; the tick in `attest_launch` is the mean over all 1,024 leaves of one identity, in `publish_metadata` the typical record). Every instruction stays far below Solana's 1.4M-CU transaction limit; the largest measured, `launch` at 221,107 CU, is about 16% of it. Hover a bar for details.

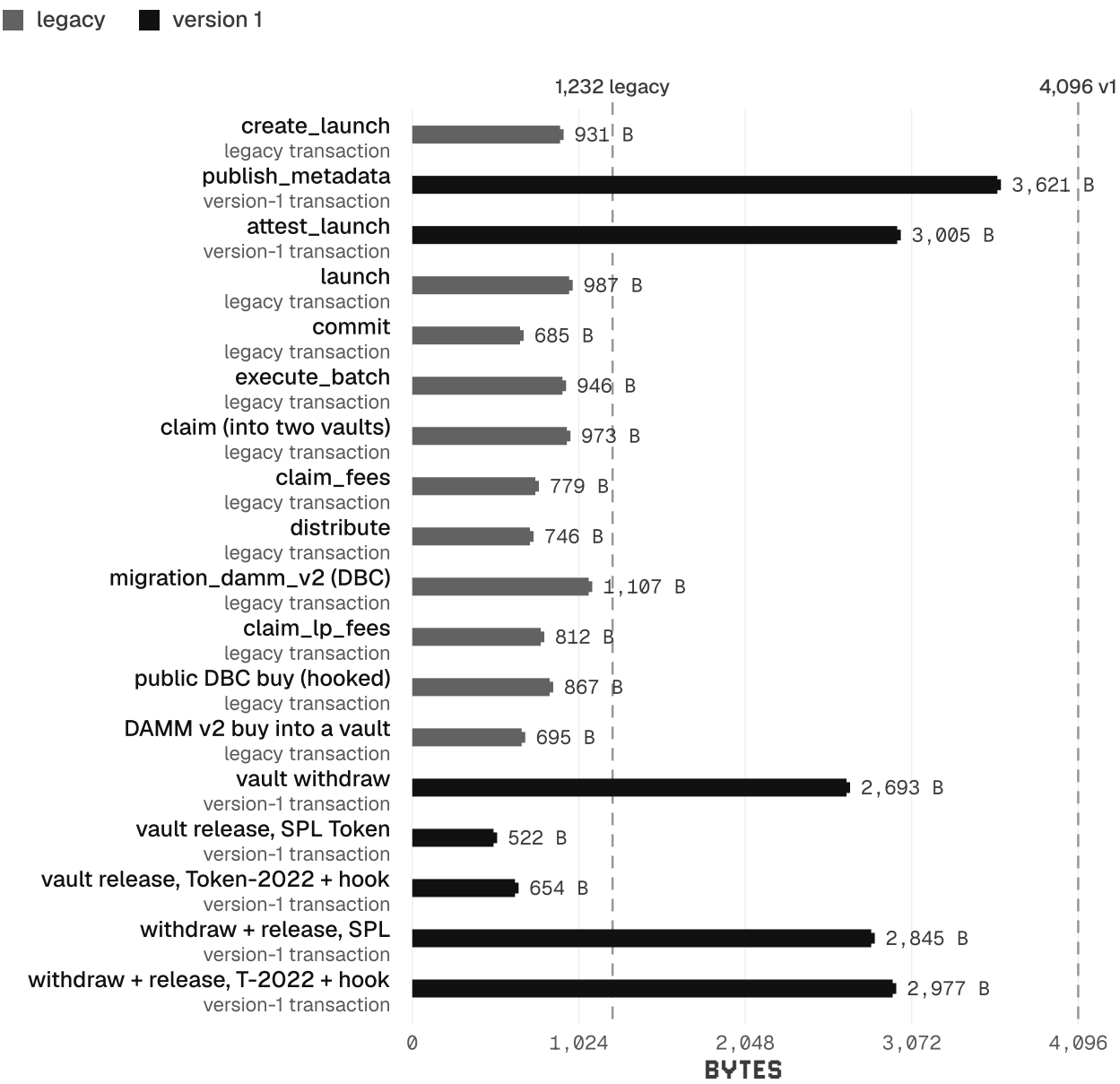


FIG. 20 **Transaction sizes against their limits.** Legacy transactions are limited to 1,232 bytes; version-1 transactions (SIMD-0385) to 4,096. Version 1 carries what cannot fit: the two hash-based signatures (an LM-OTS signature is 2,176 bytes, an LMS signature 2,508) and metadata records of up to 3,405 bytes (the bar is the largest record; the typical one is 1,460 bytes). Every wallet-signed step fits a legacy transaction, `launch` in a constant 987 bytes for every record; version-1 transactions are paid by a throwaway key, because several wallets cannot sign them.

13.1 What `launch` costs

`launch` takes no arguments and reads the record the attestation names, so its transaction is a constant 987 bytes for every record **FINAL BUILD**. Its compute is a constant base per record, measured to within 8 CU over 18 launches, plus 1,500 CU for every extra step the bump search of the six mint-derived addresses takes:

$$\mathrm{CU}_{\mathrm{launch}} = b_{\mathrm{record}} + 1,500\,e, \tag{13.1}$$

with the base b 142,672 CU with no uri, 143,996 with a 200-byte external uri and 188,107 with the largest inline uri. The extra steps e are random (seven on average); more than 47 has probability below one in a million, which bounds the worst launch at 258,607 CU with the largest inline uri and 213,172 without a uri. The SDK therefore

requests 300,000 CU; at the earlier 200,000 a launch with the largest inline uri would have run out about four times in ten. Of one inline-maximum run's 204,607 CU, DBC took 88,545 and the hook 32,085; the launchpad's own share is about 40k CU above the no-uri mode, mostly base64 (36.2k CU for a 3,072-byte document). The launchpad's heap peaks at 19,904 of its 32,768 bytes, and DBC accepts uris up to 8,685 bytes, 2.1 times the largest inline uri `FINAL BUILD`.

13.2 Other costs worth isolating

- **The batch.** An `execute_batch` on the Fair preset spent 162,504 CU, of which DBC's swap 70,156 (the hook's check of the batch buy 5,778), ending the shield 13,654, creating the escrow's token account 19,407 and the launchpad itself 59,287 `V2 BUILD`.
- **The origin.** The first attestation of a launch also creates its identity (196 bytes) and attestation (2,708 bytes) accounts: 16,057,800 lamports with the fee at mainnet's rent[6], paid by the relaying key. Its transaction is a constant 3,005 bytes; after the metadata review fix every signature also derives the record's address, which added 1,500 CU per bump-search step `FINAL BUILD`.
- **The metadata.** A typical record (1,244 bytes) holds 6,969,760 lamports of rent forever; the largest possible (3,405 bytes) 17,947,640. With the default empty uri the mint is 386 bytes plus the name and symbol; the largest inline uri makes it 4,553 bytes, still far below the runtime's 10,240-byte growth limit `FINAL BUILD`.
- **Graduation at the largest records.** `migration_damm_v2` costs 151,105 to 155,605 CU in a 1,107-byte transaction with the largest inline uri: the metadata changes nothing after launch `FINAL BUILD`.

14 Limitations and future work

14.1 The chain is not post-quantum

Noised protects origin and custody at the application level, built from `sol_sha256`. Everything else on Solana remains classical: validator and vote keys, every ordinary wallet, and the multisigs that can upgrade Meteora's programs and Token-2022. A credible quantum threat therefore needs a chain-level migration: post-quantum signature verification for transactions and accounts, a way for funds to move from Ed25519 addresses to post-quantum ones before the keys are broken, and post-quantum upgrade authorities. No such proposal is active: a Falcon verification syscall (SIMD-0461) was closed in June 2026, and no post-quantum code ships in Agave 4.3 [22]. Until then, value that must survive a quantum computer belongs in a PQ vault, and the vault's own limits apply (§10).

14.2 Wallets

Wallet support for version-1 transactions is uneven, and no mainstream wallet holds hash-based keys. Noised works around both: hash-based signatures are produced by the app from a passkey secret or a phrase and relayed by throwaway fee payers, so a user's wallet signs only legacy transactions. Wallets that hold LMS or XMSS keys natively, ideally in secure hardware as SP 800-208 envisages, would remove the browser from the signing path for both identities and vaults.

14.3 Future work

- **Independent audits, then finalization.** At least two independent audits, reproducible verified builds, and the upgrade authority of all four programs set to none. Until then the upgrade key is the largest trusted party.
- **Programmable tokens in other verifiers.** Launch rules and automation are a described component of QSL-1, shown next to the level with warnings, never a level (§12). Wallets and explorers that show the badge should show the tag and its warnings too; the SDK's `qslBadge` gives them in the standard's words.
- **Token icons outside Noised.** A token's icon and description are on-chain and signed, but third-party wallets take icons from https-hosted JSON or their own registries, so with the default empty uri they show only the name and symbol. The signed inline mode would put the document in the mint itself; it stays out of the app until a check on mainnet shows such tokens listed by wallets and indexers (one open-source indexer stores uris in a 200-character column). A creator can still submit the on-chain icon to a registry; Noised hosts nothing.
- **One-time keys across devices.** A phrase or identity used on two devices with two unshared ledgers can sign twice with one key. Authenticated record export and import narrow the window; a ledger shared by every device of a user, without a server Noised runs, is open work.
- **Stateless signatures for rare, high-value authorities.** SLH-DSA (FIPS 205) needs no state, but its smallest signature (about 7.9 KB) exceeds one transaction; verification across several transactions would lose atomicity and is left for future work.
- **Post-quantum privacy.** Confidential balances use ElGamal over Ristretto, which a quantum computer breaks retroactively. A post-quantum confidential layer is an open problem on Solana.
- **Meteora dependence.** DBC and DAMM v2 are licensed so that an immutable fork cannot be deployed. Noised discloses this dependency, monitors upgrades, and is built so that an unexpected change makes a batch refund rather than lose funds.

15 References

[1] D. McGrew, M. Curcio, S. Fluhrer. Leighton-Micali Hash-Based Signatures. RFC 8554 (IRTF), April 2019. `rfc-editor.org/rfc/rfc8554`

[2] D. Cooper, D. Apon, Q. Dang, M. Davidson, M. Dworkin, C. Miller. Recommendation for Stateful Hash-Based Signature Schemes. NIST Special Publication 800-208, October 2020. `doi.org/10.6028/NIST.SP.800-208`

[3] NIST. Stateless Hash-Based Digital Signature Standard (SLH-DSA). FIPS 205, August 2024. `doi.org/10.6028/NIST.FIPS.205`

[4] H. Krawczyk, M. Bellare, R. Canetti. HMAC: Keyed-Hashing for Message Authentication. RFC 2104, February 1997. `rfc-editor.org/rfc/rfc2104`

[5] M. Nystrom. Identifiers and Test Vectors for HMAC-SHA-224, HMAC-SHA-256, HMAC-SHA-384, and HMAC-SHA-512. RFC 4231, December 2005. `rfc-editor.org/rfc/rfc4231`

[6] S. Josefsson, I. Liusvaara. Edwards-Curve Digital Signature Algorithm (EdDSA). RFC 8032, January 2017. `rfc-editor.org/rfc/rfc8032`

[7] Solana Improvement Document SIMD-0385: version 1 transactions (4,096 bytes). Active on mainnet since slot 447,120,000 (2026-09-15). `github.com/solana-foundation/solana-improvement-documents`

[8] P. W. Shor. Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer. SIAM Journal on Computing 26(5), 1997. `doi.org/10.1137/S0097539795293172`

[9] L. K. Grover. A fast quantum mechanical algorithm for database search. Proceedings of STOC 1996. doi.org/10.1145/237814.237866

[10] J. M. Pollard. Monte Carlo methods for index computation (mod p). Mathematics of Computation 32(143), 1978. doi.org/10.2307/2006496

[11] D. J. Bernstein. Cost analysis of hash collisions: will quantum computers make SHARCS obsolete? SHARCS 2009. cr.yp.to/hash/collisioncost-20090823.pdf

[12] J. Katz. Analysis of a Proposed Hash-Based Signature Standard. Security Standardisation Research (SSR) 2016, LNCS 10074. doi.org/10.1007/978-3-319-49100-4_12

[13] R. C. Merkle. A Certified Digital Signature. Advances in Cryptology, CRYPTO '89, LNCS 435. doi.org/10.1007/0-387-34805-0_21

[14] NIST. Transition to Post-Quantum Cryptography Standards. NIST IR 8547 (initial public draft), November 2024. doi.org/10.6028/NIST.IR.8547.ipd

[15] Resource estimates for breaking 256-bit elliptic-curve discrete logarithms (about 1,200 logical qubits), arXiv:2603.28846, 2026; summarized in docs/research/FACTS.md #58. arxiv.org/abs/2603.28846

[16] W3C. Web Authentication: An API for accessing Public Key Credentials, Level 3 (PRF extension); FIDO Alliance, Client to Authenticator Protocol 2.1 (hmac-secret extension). w3.org/TR/webauthn-3

[17] M. Palatinus, P. Rusnak, A. Voisine, S. Bowe. BIP-39: Mnemonic code for generating deterministic keys, 2013. github.com/bitcoin/bips/blob/master/bip-0039.mediawiki

[18] Meteora. Dynamic Bonding Curve (program dbcij3LWUppWqq96dh6gJWwBifmcGfLSB5D4DuSMaqN, version 0.2.1): documentation, MIT-licensed SDK and IDL. docs.meteora.ag

[19] Meteora. DAMM v2 / cp_amm (program cpamdpZCGKUy5JxQXB4dcpGPiikHawvSWAd6mEn1sGG, version 0.2.4): documentation and SDK. docs.meteora.ag

[20] Solana Program Library. Token-2022 extensions: transfer hook interface, metadata, confidential transfers (v11.0.0 on mainnet). spl.solana.com/token-2022

[21] Anza. Agave validator 4.3.0: cost-model/src/block_cost_limits.rs (MAX_WRITABLE_ACCOUNT_UNITS = 24,000,000; MAX_BLOCK_UNITS = 60,000,000) and program-runtime/src/execution_budget.rs. github.com/anza-xyz/agave

[22] SIMD-0461: Falcon signature verification (closed 2026-06-17); SIMD-0579: keccak-p1600 (open). github.com/solana-foundation/solana-improvement-documents

[23] Noised engineering specification (protocol v2, safety policy 3, on-chain metadata): docs/SPEC.md, docs/POST_QUANTUM.md, docs/HOOKS.md, docs/ARCHITECTURE.md, docs/PRIVACY.md; the SDK's README and sdk/CHANGELOG.md (0.4.0). this repository

[24] Noised security documents: docs/SECURITY_GUARANTEES.md, docs/THREAT_MODEL.md, docs/AUDIT.md (internal reviews: rounds 1, 1b, 2 and 3, the PQ v2, on-chain metadata, client and SDK reviews, and the mutation testing), docs/research/FACTS.md. this repository

[25] S. Josefsson. The Base16, Base32, and Base64 Data Encodings. RFC 4648, October 2006. rfc-editor.org/rfc/rfc4648

[26] W3C. Portable Network Graphics (PNG) Specification (Third Edition), 2025. w3.org/TR/png-3

[27] Metaplex Foundation. Digital Asset RPC infrastructure (the open-source DAS indexer): init.sql (metadata_url varchar(200)) and program_transformers/src/token_extensions; commit 9c7e1b6, 2026-09-17; with the wallet and

indexer findings in docs/research/FACTS.md. `github.com/metaplex-foundation/digital-asset-rpc-infrastructure`

Notes

1. Bits of security here means log2 of the expected number of basic operations (group operations, hash evaluations or Grover iterations) of the best known generic attack. It ignores memory, parallelism and error correction, which make quantum attacks costlier in practice, never cheaper. ↩
2. The generic quantum attack on a 256-bit hash target is a Grover search over 2^256, about 2^128 iterations, the same order as a quantum key search on AES-256. That comparison is all we claim: the LMS and LM-OTS parameter sets are defined in RFC 8554 and listed in SP 800-208, but Noised's keys are software keys from a phrase or a passkey secret, outside the hardware setting SP 800-208 approves, so no SP 800-208 approval is claimed (docs/POST_QUANTUM.md §3.2). ↩
3. An RFC 8032 secret key is a 32-byte seed k. SHA-512(k) yields the scalar a (first half, clamped) and a nonce prefix (second half). The public key is A = [a]B. Shor's algorithm returns a modulo the group order, which suffices to sign anything, but it does not return the prefix: recovering it means finding a SHA-512 preimage. The distinction matters for wallet-derived secrets (section 5). ↩
4. Each floor loses less than one base unit per commit, so the escrow keeps at most n - 1 base units and n - 1 lamports of dust for n commits (in practice fewer). The dust is never paid out. ↩
5. 400 ms is the nominal slot time; real slots vary. The model is linear in the number of slots, so a different slot time scales every curve by the same factor. ↩
6. Rent-exempt balances are (128 + size) x 5,080 lamports at mainnet's rate (docs/research/FACTS.md). LiteSVM's default rate, used by some measurements in section 13, is 6,960 lamports per byte; where a figure comes from LiteSVM the text says so. ↩
7. Grover's algorithm parallelizes badly: splitting the search over M machines only reduces the time by a factor sqrt(M). A 2^128-iteration search is also long and serial, so these figures are lower bounds on the real cost. ↩